DETAILS

NAME:TATAPUDI SAI SUSHANTH VARMA

SECTION:CSE 1

ROLL NO:61

Case 1: University Scholarship Allocation

Problem Statement

Evaluate applicants using aggregate marks, subject wise minimum marks, attendance, and family income. Rank eligible students and apply predefined tie breaking rules.

Objective

Identify eligible applicants and rank them by total marks; break ties by attendance, then lower family income.

Assumptions

- Subjects: Math, English, Science.

- The user enters each subject’s minimum mark, minimum attendance, and maximum income.

- Applicants must meet all criteria.

- Ties are resolved by higher attendance, then lower income, then name alphabetically.

Algorithm

1. Read the eligibility thresholds and number of applicants.

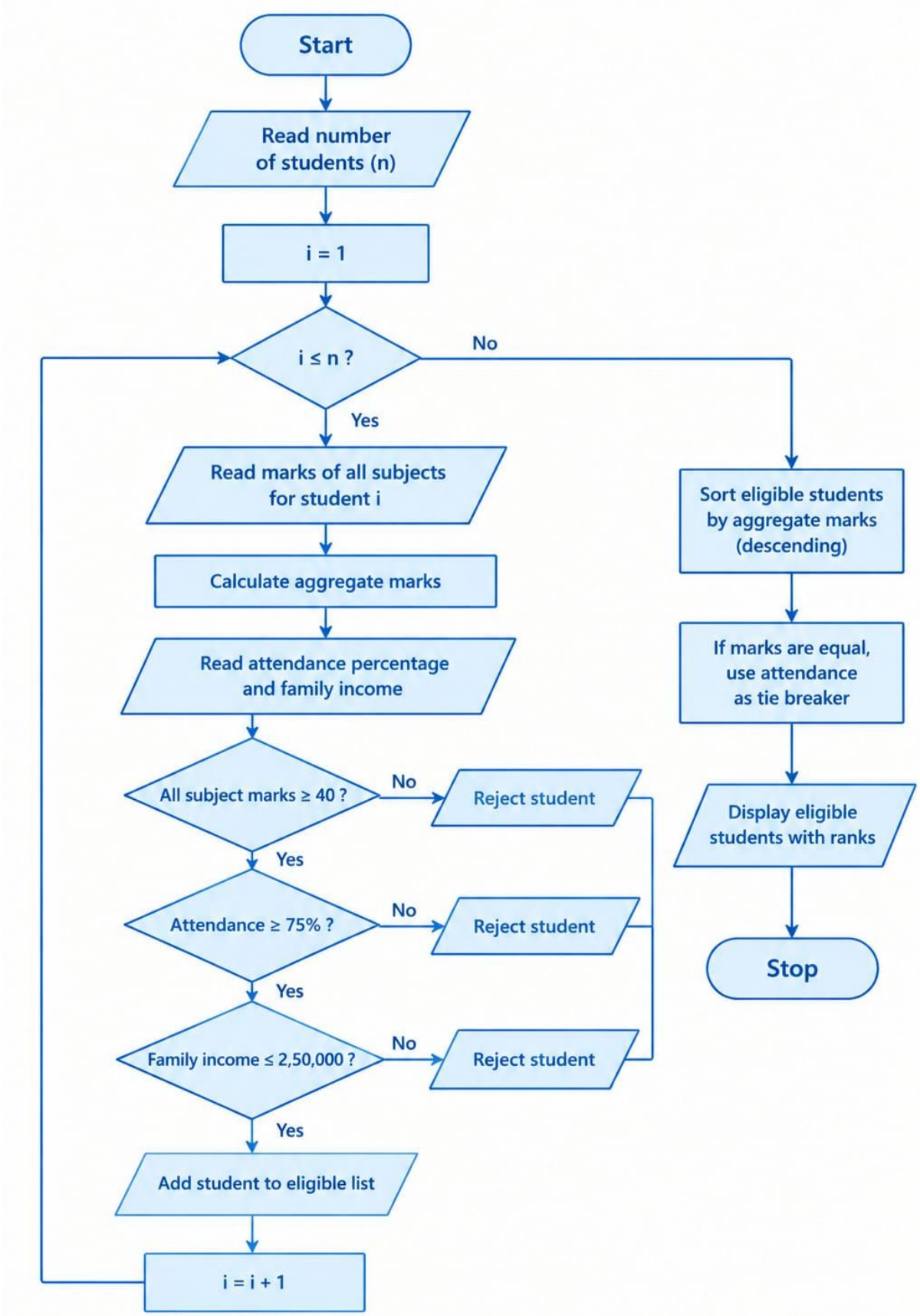
2. For each applicant, read marks, attendance, and family income.

3. Check that every subject mark and both other criteria meet the thresholds.

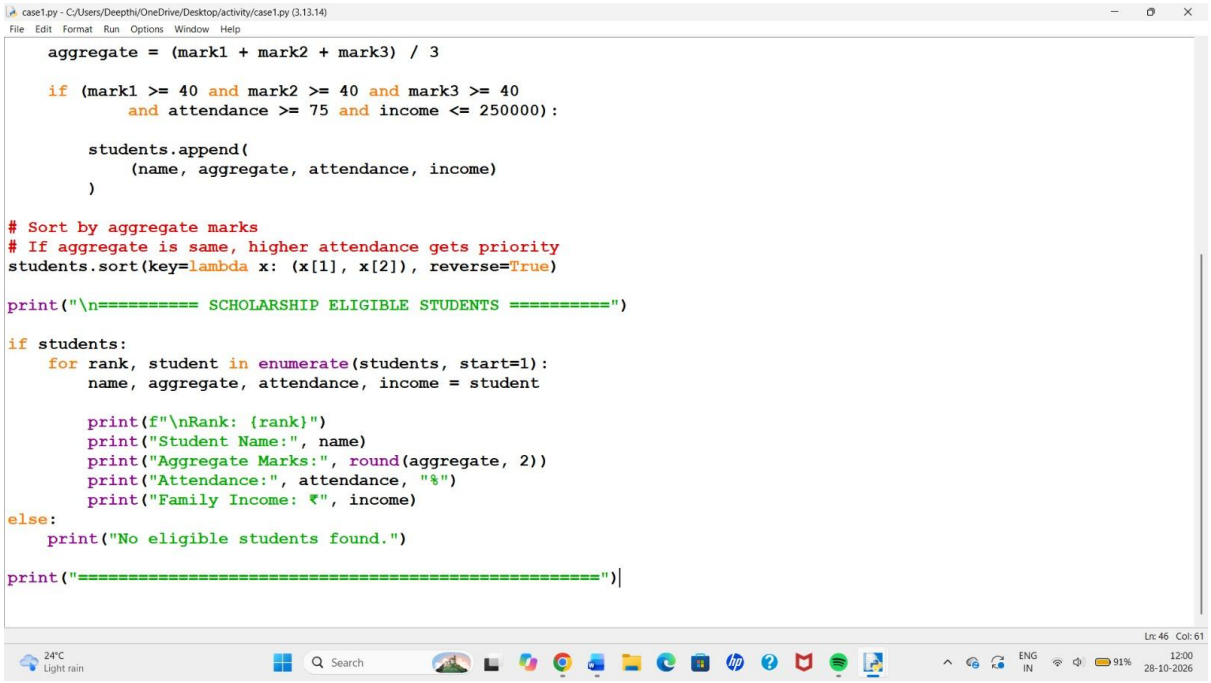
4. Store eligible applicants.

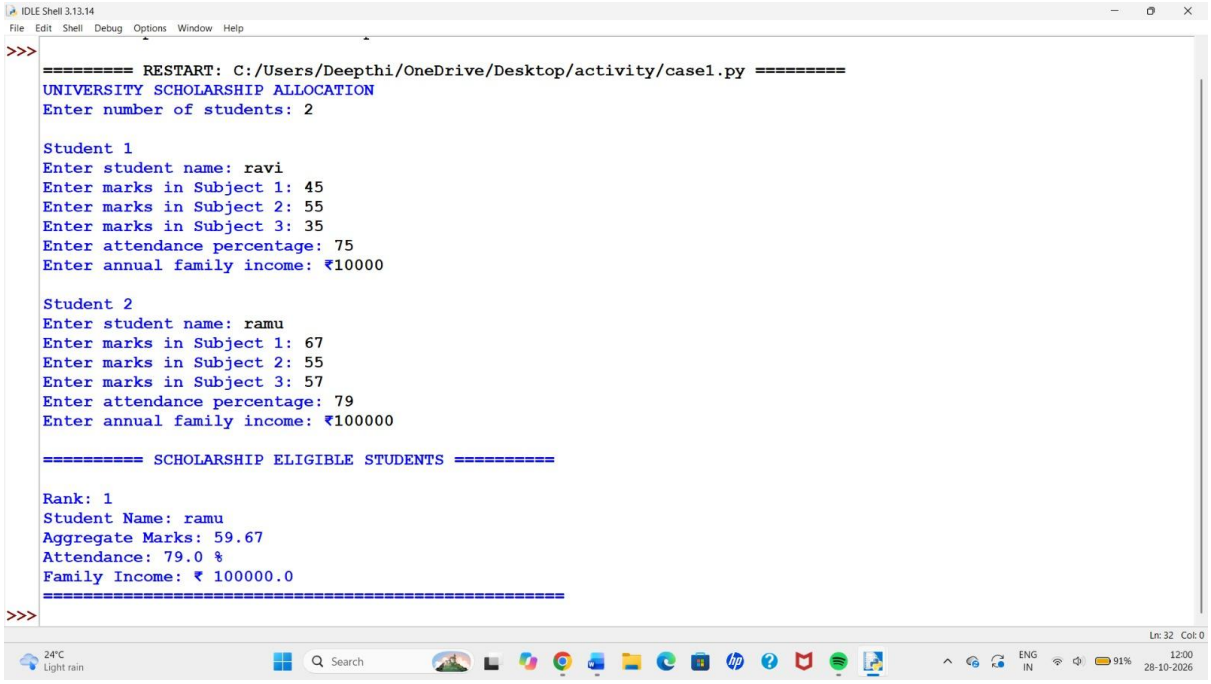
5. Sort eligible applicants by the tie breaking rules.
6. Display the ranked list.

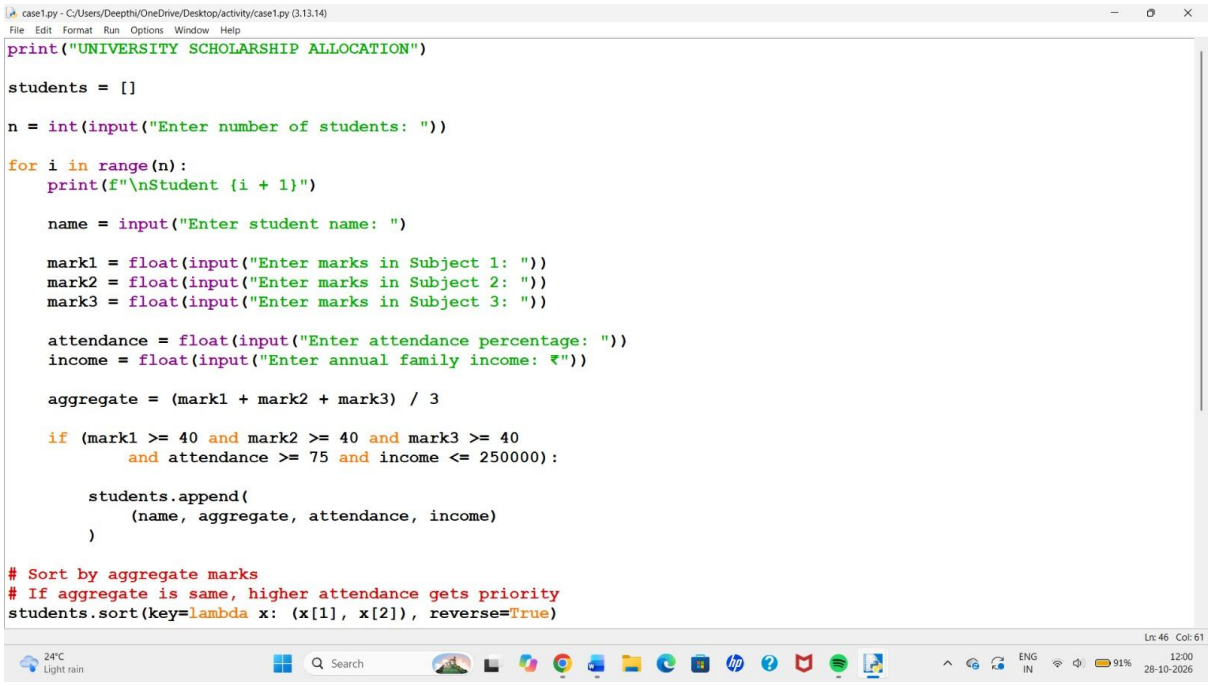
Flowchart


Start
Read number of students (n)
i = 1
i ≤ n ?
No
Yes
Read marks of all subjects for student i
Calculate aggregate marks
Read attendance percentage and family income
All subject marks ≥ 40 ?
No
Reject student
Yes
Attendance ≥ 75% ?
No
Reject student
Yes
Family income ≤ 2,50,000 ?
No
Reject student
Yes
Add student to eligible list
i = i + 1
Sort eligible students by aggregate marks (descending)
If marks are equal, use attendance as tie breaker
Display eligible students with ranks
Stop

Pseudocode


case1.py - C:/Users/Deepthi/OneDrive/Desktop/activity/case1.py (3.13.14)
File Edit Format Run Options Window Help
aggregate = (mark1 + mark2 + mark3) / 3
if (mark1 >= 40 and mark2 >= 40 and mark3 >= 40
and attendance >= 75 and income <= 250000):
students.append(
(name, aggregate, attendance, income)
)
Sort by aggregate marks
If aggregate is same, higher attendance gets priority
students.sort(key=lambda x: (x[1], x[2]), reverse=True)
print("\n========== SCHOLARSHIP ELIGIBLE STUDENTS ==========")
if students:
for rank, student in enumerate(students, start=1):
name, aggregate, attendance, income = student
print(f"\nRank: {rank}")
print("Student Name:", name)
print("Aggregate Marks:", round(aggregate, 2))
print("Attendance:", attendance, "%")
print("Family Income: ₹", income)
else:
print("No eligible students found.")
print("==")
Ln: 46 Col: 61


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
>>>
========= RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/case1.py =========
UNIVERSITY SCHOLARSHIP ALLOCATION
Enter number of students: 2
Student 1
Enter student name: ravi
Enter marks in Subject 1: 45
Enter marks in Subject 2: 55
Enter marks in Subject 3: 35
Enter attendance percentage: 75
Enter annual family income: ₹10000
Student 2
Enter student name: ramu
Enter marks in Subject 1: 67
Enter marks in Subject 2: 55
Enter marks in Subject 3: 57
Enter attendance percentage: 79
Enter annual family income: ₹100000
========== SCHOLARSHIP ELIGIBLE STUDENTS ==========
Rank: 1
Student Name: ramu
Aggregate Marks: 59.67
Attendance: 79.0 %
Family Income: ₹ 100000.0
==
>>>
Ln: 32 Col: 0



INPUT subject minimums, attendance minimum, income maximum

INPUT number of applicants

eligible = empty list

FOR each applicant

INPUT name, subject marks, attendance, income

IF all marks meet minimums AND attendance meets minimum

AND income is within maximum

ADD applicant to eligible

ENDIF

ENDFOR

SORT eligible by marks descending, attendance descending,

income ascending, name ascending

DISPLAY eligible

Python Code

```
subjects = ["Math", "English", "Science"]
```

```
minimums = [float(input(f"Minimum mark for {s}: ")) for s in subjects]
min_attendance = float(input("Minimum attendance (%): "))
max_income = float(input("Maximum family income: "))
students = []

count = int(input("Number of applicants: "))

for _ in range(count):
    print("\nApplicant details")
    name = input("Name: ")
```

```
    marks = [float(input(f"{s} mark: ")) for s in subjects]
    attendance = float(input("Attendance (%): "))
    income = float(input("Family income: "))

    meets_marks = all(marks[i] >= minimums[i] for i in range(len(subjects)))
    if meets_marks and attendance >= min_attendance and income <= max_income:
        students.append([name, sum(marks), attendance, income])
```

```
students.sort(key=lambda x: (-x[1], -x[2], x[3], x[0].lower()))


print("\nEligible applicants, ranked")
if not students:
    print("No eligible applicants.")
else:
    for rank, student in enumerate(students, start=1):
        print(
            f"{rank}. {student[0]} | Total marks: {student[1]:.1f} "
            f"| Attendance: {student[2]:.1f}% | Income: {student[3]:.2f}"
```

)

Test Case and Expected Output

Assume minimum marks are 60 in each subject, attendance minimum is 75%, and income maximum is 300,000. Applicant Asha has marks 80, 75, 90, attendance 90%, income 250,000. Applicant Bala has marks 85, 85, 75, attendance 80%, income 280,000.

Eligible applicants, ranked

1. Asha | Total marks: 245.0 | Attendance: 90.0% | Income: 250000.00

2. Bala | Total marks: 245.0 | Attendance: 80.0% | Income: 280000.00

Conclusion

The program filters applicants against all the entered criteria and ranks eligible students using the specified tie breaking rules.

Case 2: Progressive Electricity Billing

Problem Statement

Calculate household electricity bills using consumption slabs, fixed charges, surcharges, and rebates. Compare bills across months and identify unusual increases.

Objective

Calculate each monthly bill and flag a bill that is more than 50% higher than the previous month.

Assumptions

- First 100 units cost 2 per unit.
- Next 100 units cost 3 per unit.
- Units above 200 cost 5 per unit.

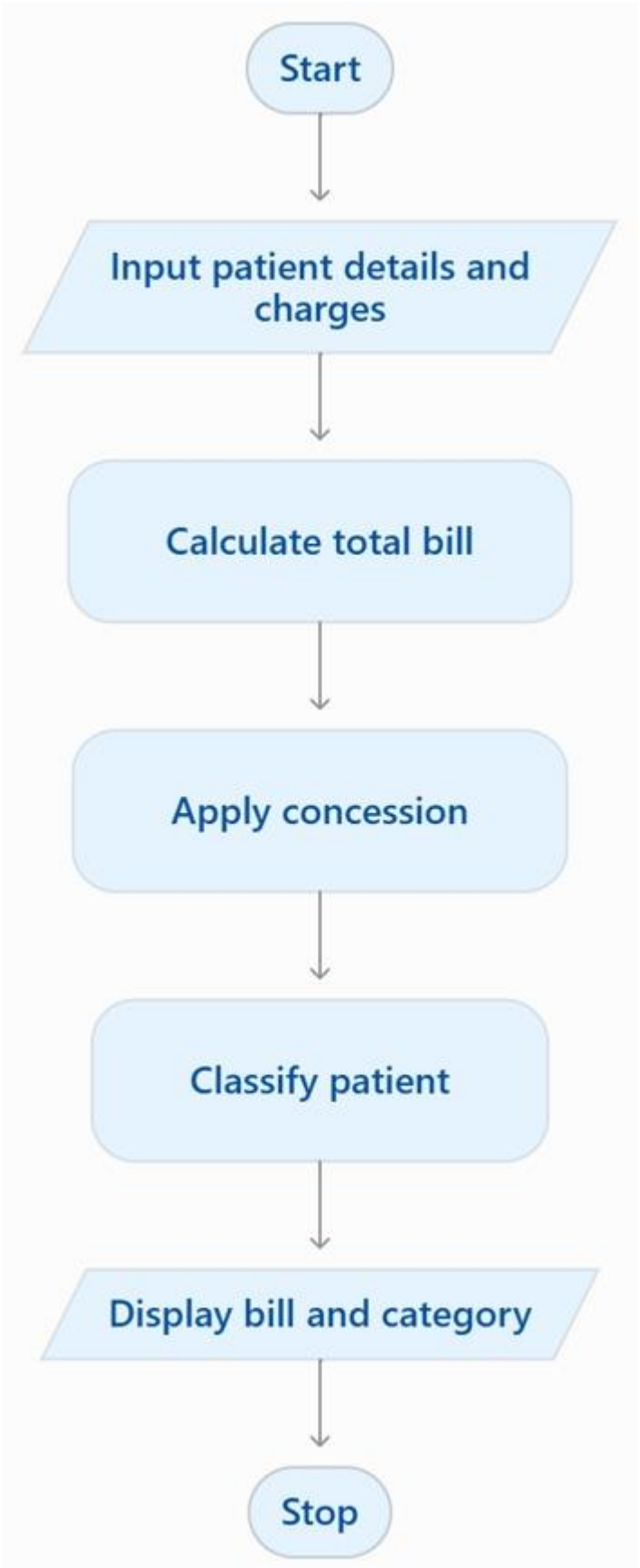
- Fixed charge is 50 per month.
- A 5% surcharge applies when the bill before surcharge exceeds 500.
- A 2% rebate applies when usage is below 100 units.
- An unusual increase means the current bill is more than 1.5 times the previous bill.

Algorithm

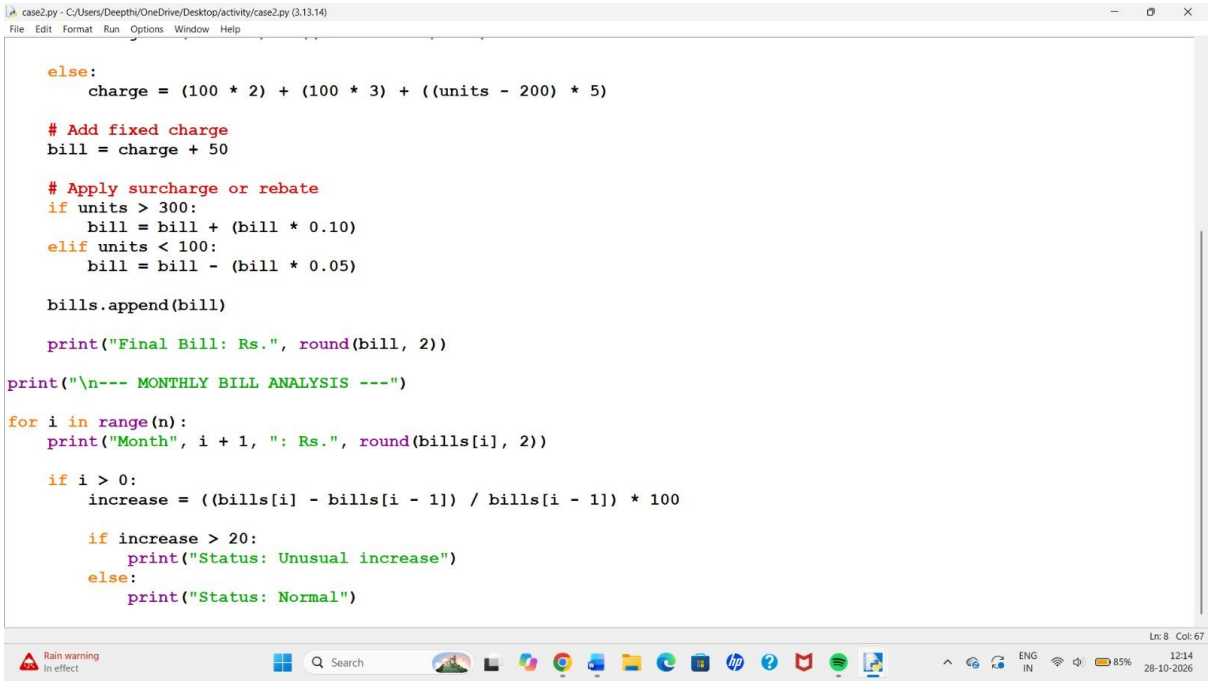
1. Read the number of months.
2. For each month, read units consumed.
3. Calculate the energy charge by slab.
4. Add the fixed charge and apply any surcharge or rebate.

5. Compare with the previous month and display a warning if the increase is unusual.

Flowchart


Start
Input patient details and charges
Calculate total bill
Apply concession
Classify patient
Display bill and category
Stop

Pseudocode



```
    else:
        charge = (100 * 2) + (100 * 3) + ((units - 200) * 5)

    # Add fixed charge
    bill = charge + 50

    # Apply surcharge or rebate
    if units > 300:
        bill = bill + (bill * 0.10)
    elif units < 100:
        bill = bill - (bill * 0.05)

    bills.append(bill)

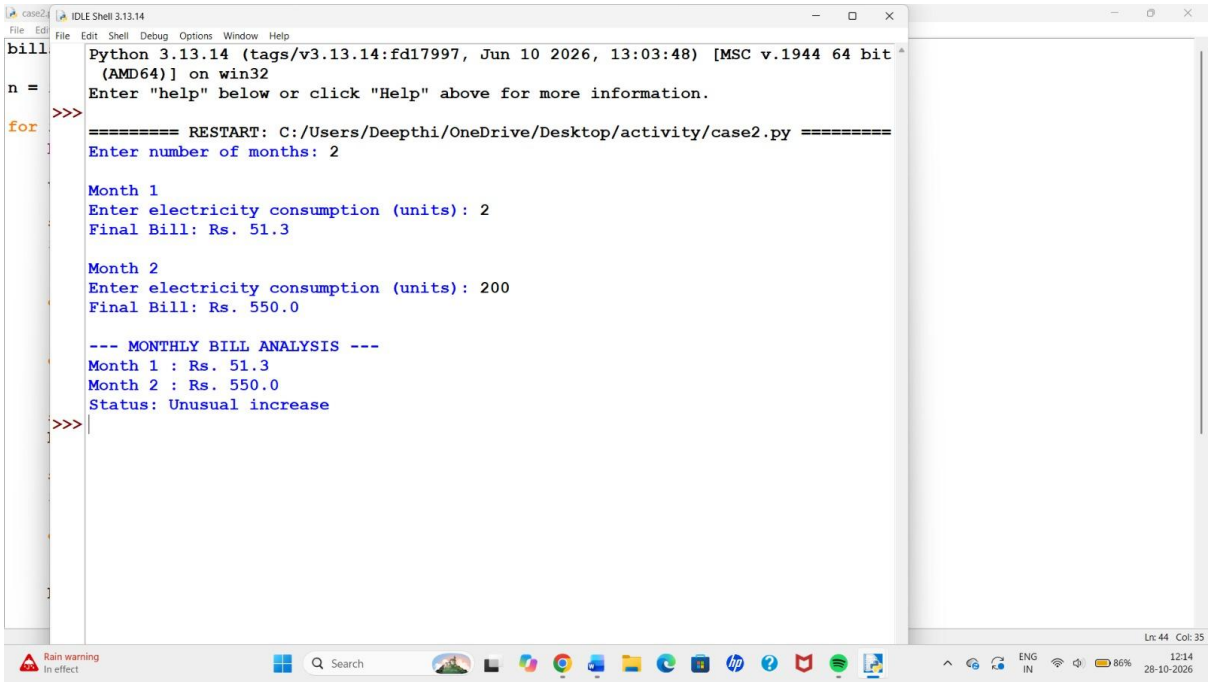
    print("Final Bill: Rs.", round(bill, 2))

print("\n--- MONTHLY BILL ANALYSIS ---")

for i in range(n):
    print("Month", i + 1, ": Rs.", round(bills[i], 2))

    if i > 0:
        increase = ((bills[i] - bills[i - 1]) / bills[i - 1]) * 100

        if increase > 20:
            print("Status: Unusual increase")
        else:
            print("Status: Normal")
```

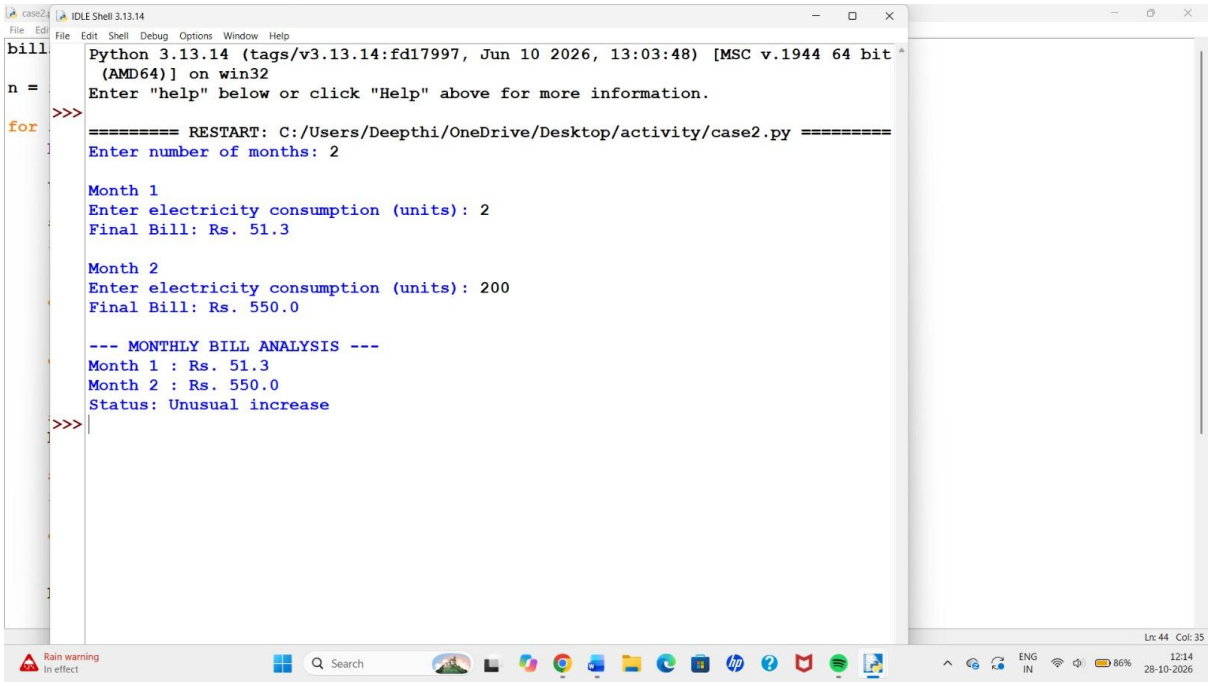


```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/case2.py =========
Enter number of months: 2

Month 1
Enter electricity consumption (units): 2
Final Bill: Rs. 51.3

Month 2
Enter electricity consumption (units): 200
Final Bill: Rs. 550.0

--- MONTHLY BILL ANALYSIS ---
Month 1 : Rs. 51.3
Month 2 : Rs. 550.0
Status: Unusual increase
>>>
```



INPUT number of months

previous_bill = none

FOR each month

INPUT units

CALCULATE slab charge

bill = slab charge + fixed charge

IF bill > surcharge threshold

```
        ADD surcharge
    ELSE IF units < rebate threshold
        APPLY rebate
    ENDIF

    DISPLAY bill
    IF previous_bill exists AND bill > 1.5
* previous_bill
        DISPLAY unusual increase
warning
    ENDIF
    previous_bill = bill
ENDFOR
```

Python Code

```
months = int(input("Number of months: "))
previous_bill = None

for month in range(1, months + 1):
    units = float(input(f"Units used in month {month}: "))

    if units < 0:
        print("Units cannot be negative.")
        continue

    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
```

```
        energy_charge = 100 * 2 + (units - 100) * 3
    else:
        energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = energy_charge + 50

    if bill > 500:
        bill *= 1.05
    elif units < 100:
        bill *= 0.98

    print(f"Month {month} bill: {bill:.2f}")
```

```
    if previous_bill is not None and bill
> previous_bill * 1.5:
        print("Warning: unusual increase
from the previous month.")


    previous_bill = bill
```

Test Case and Expected Output

Input usage: 80 units, then 180 units.

Month 1 bill: 206.80

Month 2 bill: 390.00

No warning is displayed because 390 is not more than 1.5 times 206.80.

Conclusion

The program uses progressive slabs to calculate monthly bills, applies the stated adjustments, and identifies large month to month increases.

Case 3: Hospital Billing and Patient Classification

Problem Statement

Process consultation, diagnostic, and treatment charges; apply concessions; validate billing constraints; and classify patients according to payment status.

Objective

Calculate each patient’s final bill, remaining balance, and payment classification.

Assumptions

- Patients aged 60 or above receive a 10% concession.

- Otherwise, patients with monthly income below 20,000 receive a 15% concession.

- Only one concession is applied.

- Payment cannot be negative or exceed the final bill.

- Payment status is unpaid, partially paid, or paid.

Algorithm

1. Read the number of patients.

2. For each patient, read identifying and billing details.

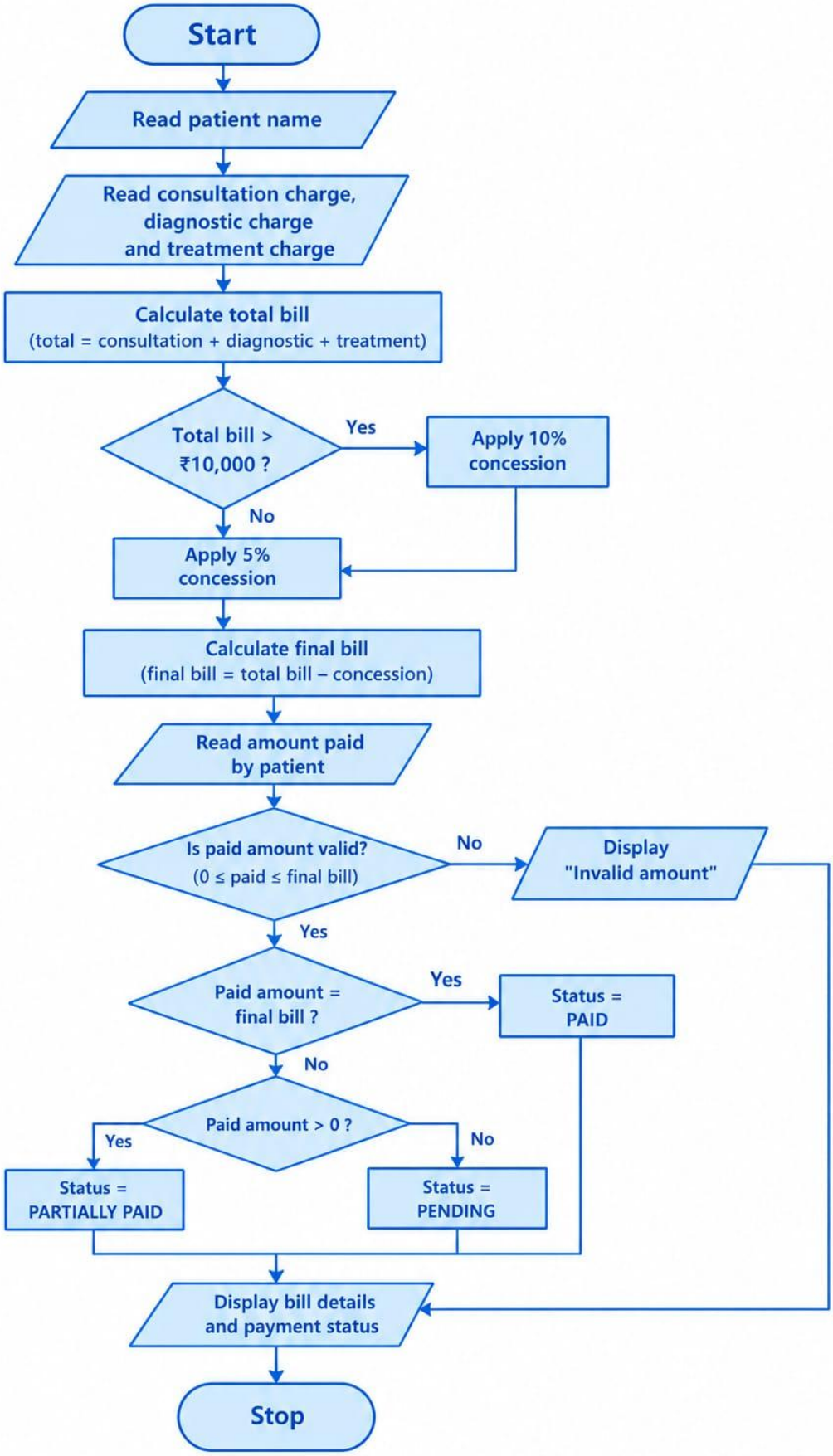
3. Validate that charges and payment are nonnegative.

4. Calculate gross charges and apply the applicable concession.

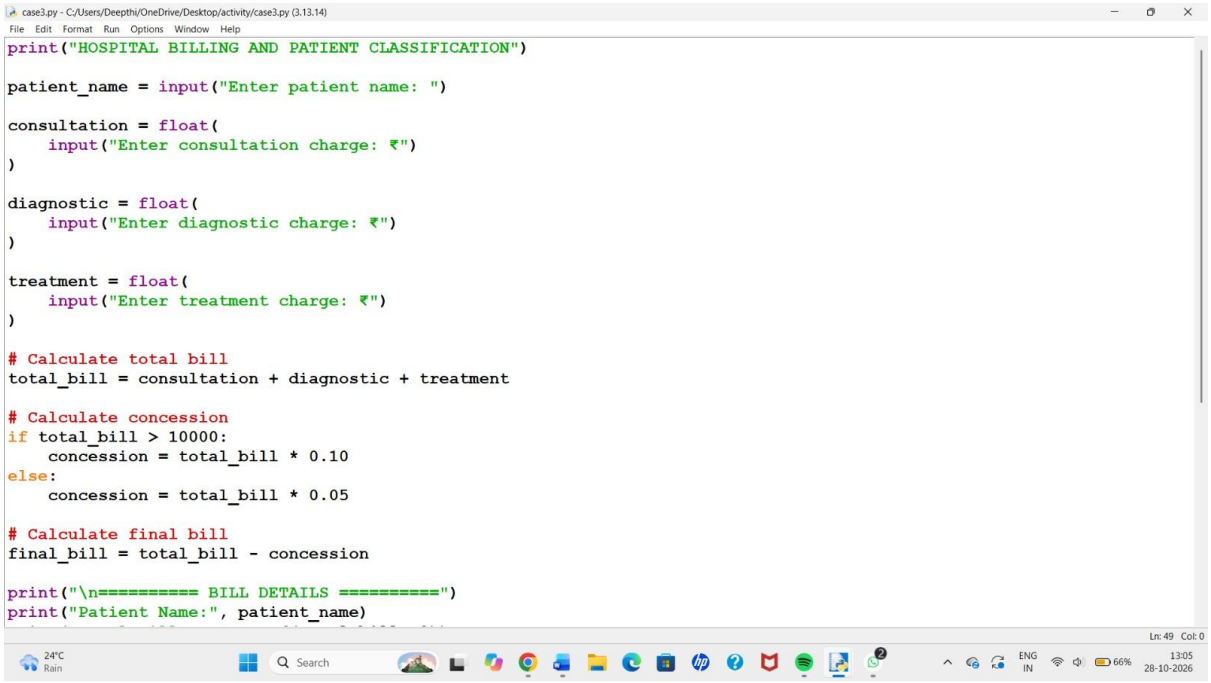
5. Validate payment against the final bill.

6. Calculate the balance and display patient category and payment status.

Flowchart


Start
Read patient name
Read consultation charge, diagnostic charge and treatment charge
Calculate total bill (total = consultation + diagnostic + treatment)
Total bill > ₹10,000 ?
Yes
Apply 10% concession
No
Apply 5% concession
Calculate final bill (final bill = total bill – concession)
Read amount paid by patient
Is paid amount valid? (0 ≤ paid ≤ final bill)
No
Display "Invalid amount"
Yes
Paid amount = final bill ?
Yes
Status = PAID
No
Paid amount > 0 ?
Yes
Status = PARTIALLY PAID
No
Status = PENDING
Display bill details and payment status
Stop

Pseudocode



```
print("HOSPITAL BILLING AND PATIENT CLASSIFICATION")

patient_name = input("Enter patient name: ")

consultation = float(
    input("Enter consultation charge: ₹")
)

diagnostic = float(
    input("Enter diagnostic charge: ₹")
)

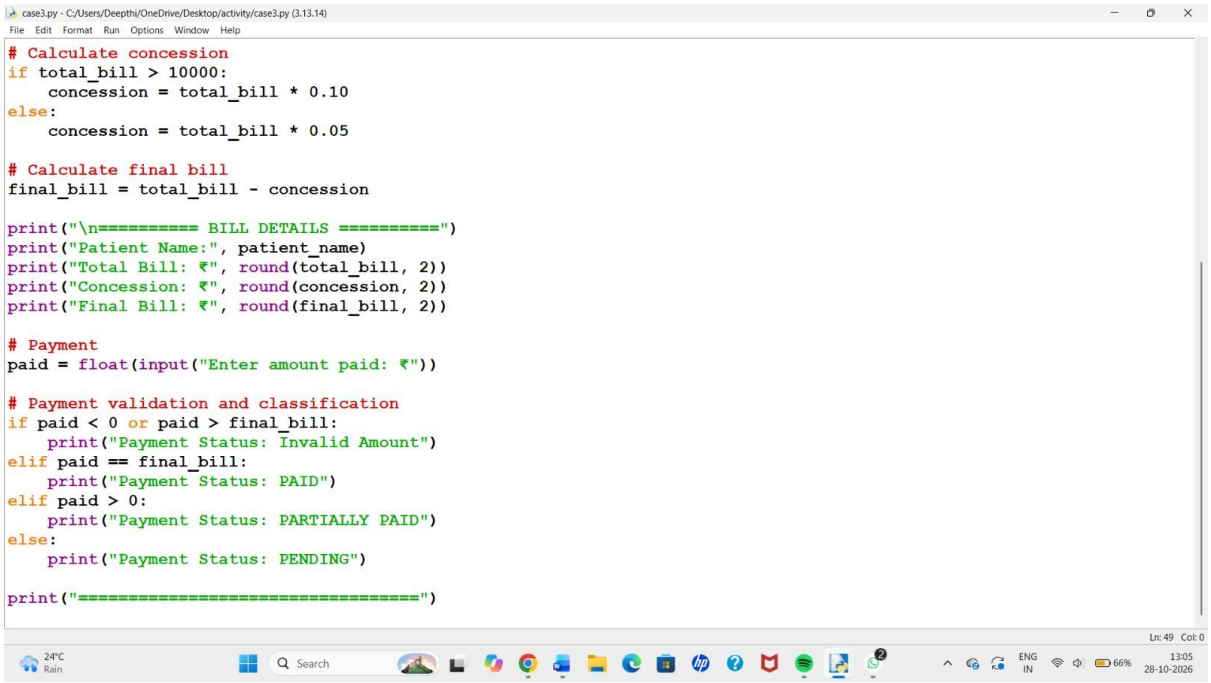
treatment = float(
    input("Enter treatment charge: ₹")
)

# Calculate total bill
total_bill = consultation + diagnostic + treatment

# Calculate concession
if total_bill > 10000:
    concession = total_bill * 0.10
else:
    concession = total_bill * 0.05

# Calculate final bill
final_bill = total_bill - concession

print("\n========== BILL DETAILS ==========")
print("Patient Name:", patient_name)
```



```
# Calculate concession
if total_bill > 10000:
    concession = total_bill * 0.10
else:
    concession = total_bill * 0.05

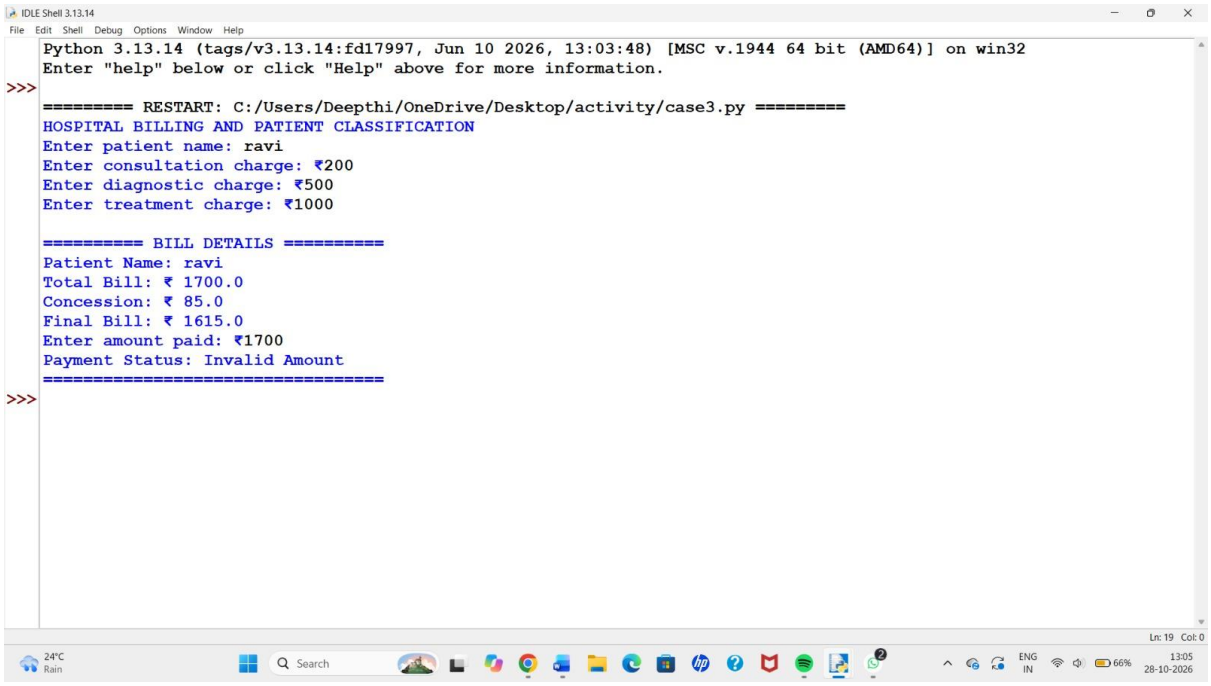
# Calculate final bill
final_bill = total_bill - concession

print("\n========== BILL DETAILS ==========")
print("Patient Name:", patient_name)
print("Total Bill: ₹", round(total_bill, 2))
print("Concession: ₹", round(concession, 2))
print("Final Bill: ₹", round(final_bill, 2))

# Payment
paid = float(input("Enter amount paid: ₹"))

# Payment validation and classification
if paid < 0 or paid > final_bill:
    print("Payment Status: Invalid Amount")
elif paid == final_bill:
    print("Payment Status: PAID")
elif paid > 0:
    print("Payment Status: PARTIALLY PAID")
else:
    print("Payment Status: PENDING")

print("==================================")
```



INPUT number of patients

FOR each patient

INPUT name, age, income, charges, payment

IF any charge or payment is negative

DISPLAY invalid input

ELSE

```
total = sum of charges
IF age >= 60
   concession = 10% of total
ELSE IF income < 20000
   concession = 15% of total
ELSE
   concession = 0
ENDIF

bill = total - concession
IF payment > bill
   DISPLAY invalid payment
ELSE
   balance = bill - payment
```

CLASSIFY as paid, partially paid, or unpaid

DISPLAY bill and classification

ENDIF

ENDIF

ENDFOR

Python Code

```
count = int(input("Number of patients: "))


for _ in range(count):
    print("\nPatient details")
    name = input("Name: ")
    age = int(input("Age: "))
```

```
    income = float(input("Monthly
income: "))
    consultation =
float(input("Consultation charge: "))
    diagnostic = float(input("Diagnostic
charge: "))
    treatment = float(input("Treatment
charge: "))
    paid = float(input("Amount paid: "))


    values = [income, consultation,
diagnostic, treatment, paid]
    if min(values) < 0:
        print("Invalid: charges, income,
and payment cannot be negative.")
```

```
        continue

    total = consultation + diagnostic +
treatment

    if age >= 60:
        concession_rate = 0.10
        category = "Senior"
    elif income < 20000:
        concession_rate = 0.15
        category = "Concession"
    else:
        concession_rate = 0
        category = "Regular"
```

```
    bill = total * (1 - concession_rate)

    if paid > bill:
        print("Invalid: payment cannot exceed the final bill.")
        continue

    balance = bill - paid
    if balance == 0:
        status = "Paid"
    elif paid == 0:
        status = "Unpaid"
    else:
        status = "Partially paid"
```

```
    print(f"{name} | Category: {category} | Final bill: {bill:.2f}")
    print(f"Paid: {paid:.2f} | Balance: {balance:.2f} | Status: {status}")
```

Test Case and Expected Output

A patient aged 65 has charges of 500, 1,000, and 2,500, and pays 2,000. The 10% senior concession applies.

Ravi | Category: Senior | Final bill: 3600.00

Paid: 2000.00 | Balance: 1600.00 | Status: Partially paid

Conclusion

The program calculates patient bills, applies the selected concession,

checks payment validity, and reports each patient’s outstanding balance.

Case 4: Employee Payroll and Deduction Analysis

Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives. Apply progressive deductions and produce a formatted salary statement.

Objective

Calculate gross salary, income tax, other deductions, and take home salary.

Assumptions

- Gross salary is basic pay plus allowances, overtime pay, and incentive.

- Tax is 0% on the first 20,000, 10% on the next 30,000, and 20% on income above 50,000.

- Other deductions are entered by the user.

- All entered values must be nonnegative.

Algorithm

1. Read employee salary details.

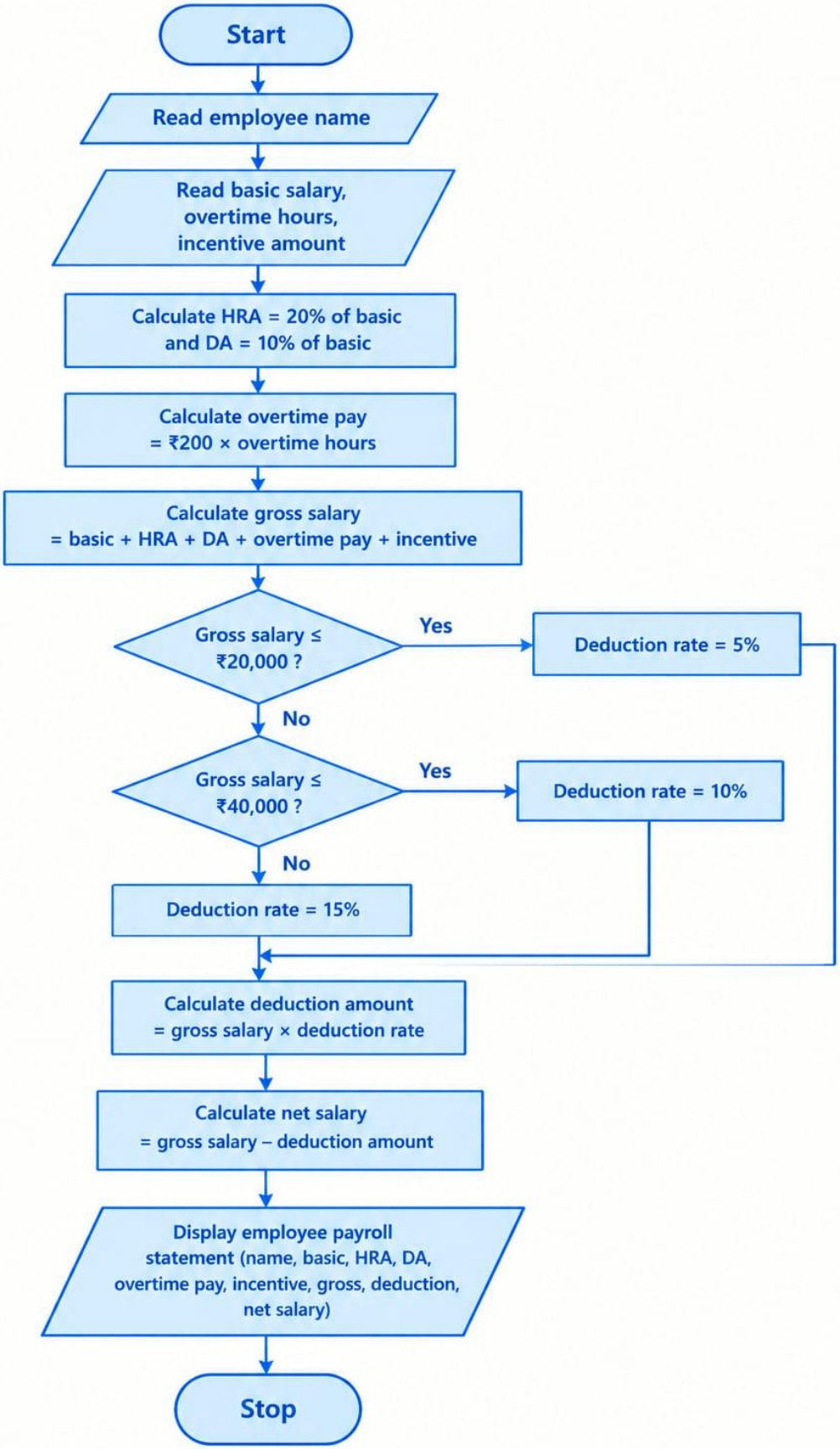
2. Calculate overtime pay and gross salary.

3. Calculate progressive tax according to the example brackets.

4. Add other deductions.

5. Calculate net salary and display the salary statement.

Flowchart


Start
Read employee name
Read basic salary, overtime hours, incentive amount
Calculate HRA = 20% of basic and DA = 10% of basic
Calculate overtime pay = ₹200 × overtime hours
Calculate gross salary = basic + HRA + DA + overtime pay + incentive
Gross salary ≤ ₹20,000 ?
Yes
Deduction rate = 5%
No
Gross salary ≤ ₹40,000 ?
Yes
Deduction rate = 10%
No
Deduction rate = 15%
Calculate deduction amount = gross salary × deduction rate
Calculate net salary = gross salary – deduction amount
Display employee payroll statement (name, basic, HRA, DA, overtime pay, incentive, gross, deduction, net salary)
Stop

Pseudocode



```
# Calculate gross salary
gross_salary = basic + hra + da + overtime_pay + incentive

# Calculate progressive deduction
if gross_salary <= 20000:
    deduction_rate = 0.05
elif gross_salary <= 40000:
    deduction_rate = 0.10
else:
    deduction_rate = 0.15

deduction = gross_salary * deduction_rate

# Calculate net salary
net_salary = gross_salary - deduction

# Salary statement
print("\n========== SALARY STATEMENT ==========")
print("Employee Name :", name)
print("Basic Pay     : Rs.", round(basic, 2))
print("HRA           : Rs.", round(hra, 2))
print("DA            : Rs.", round(da, 2))
print("Overtime Pay  : Rs.", round(overtime_pay, 2))
print("Incentive     : Rs.", round(incentive, 2))
print("--------------------------------------")
print("Gross Salary  : Rs.", round(gross_salary, 2))
print("Deduction     : Rs.", round(deduction, 2))
print("Net Salary    : Rs.", round(net_salary, 2))
print("======================================")
```



```
print("EMPLOYEE PAYROLL SYSTEM")

name = input("Enter employee name: ")
basic = float(input("Enter basic pay: "))
overtime_hours = float(input("Enter overtime hours: "))
incentive = float(input("Enter incentive amount: "))

# Calculate allowances
hra = basic * 0.20
da = basic * 0.10

# Calculate overtime pay
overtime_pay = overtime_hours * 200

# Calculate gross salary
gross_salary = basic + hra + da + overtime_pay + incentive

# Calculate progressive deduction
if gross_salary <= 20000:
    deduction_rate = 0.05
elif gross_salary <= 40000:
    deduction_rate = 0.10
else:
    deduction_rate = 0.15

deduction = gross_salary * deduction_rate

# Calculate net salary
net_salary = gross_salary - deduction
```



```
print("EMPLOYEE PAYROLL SYSTEM")

name = input("Enter employee name: ")
basic = float(input("Enter basic pay: "))
overtime_hours = float(input("Enter overtime hours: "))
incentive = float(input("Enter incentive amount: "))

# Calculate allowances
hra = basic * 0.20
da = basic * 0.10

# Calculate overtime pay
overtime_pay = overtime_hours * 200

# Calculate gross salary
gross_salary = basic + hra + da + overtime_pay + incentive

# Calculate progressive deduction
if gross_salary <= 20000:
    deduction_rate = 0.05
elif gross_salary <= 40000:
    deduction_rate = 0.10
else:
    deduction_rate = 0.15

deduction = gross_salary * deduction_rate

# Calculate net salary
net_salary = gross_salary - deduction
```

INPUT employee name, basic, allowances, overtime, rate, incentive, other deductions

gross = basic + allowances + overtime * rate + incentive

IF gross <= 20000

    tax = 0

ELSE IF gross <= 50000

    tax = 10% of amount above 20000

ELSE

    tax = 10% of 30000 + 20% of amount above 50000

ENDIF

net = gross - tax - other deductions

DISPLAY formatted salary statement

Python Code

```
name = input("Employee name: ")
basic = float(input("Basic pay: "))
allowances =
float(input("Allowances: "))
overtime_hours =
float(input("Overtime hours: "))
overtime_rate = float(input("Overtime
rate per hour: "))
incentive = float(input("Incentive: "))
other_deductions =
float(input("Other deductions: "))
```

```
inputs = [basic, allowances,
overtime_hours, overtime_rate,
        incentive, other_deductions]

if min(inputs) < 0:
    print("Invalid: payroll amounts and
hours cannot be negative.")
else:
    overtime_pay = overtime_hours *
overtime_rate
    gross = basic + allowances +
overtime_pay + incentive

    if gross <= 20000:
        tax = 0
```

```
    elif gross <= 50000:
        tax = (gross - 20000) * 0.10
    else:
        tax = 30000 * 0.10 + (gross - 50000) * 0.20

    total_deductions = tax + other_deductions
    net_salary = gross - total_deductions

    print(f"\nSalary Statement: {name}")
    print(f"Basic pay:        {basic:.2f}")
```

```
    print(f"Allowances:
{allowances:.2f}")
    print(f"Overtime pay:
{overtime_pay:.2f}")
    print(f"Incentive:
{incentive:.2f}")
    print(f"Gross salary:      {gross:.2f}")
    print(f"Income tax:        {tax:.2f}")
    print(f"Other deductions:
{other_deductions:.2f}")
    print(f"Net salary:
{net_salary:.2f}")
```

Test Case and Expected Output

Inputs: basic pay 40,000; allowances 5,000; 10 overtime hours at 200/hour;

incentive 1,000; other deductions 1,000.

Gross salary: 48000.00

Income tax: 2800.00

Other deductions: 1000.00

Net salary: 44200.00

Conclusion

The program combines salary components, calculates progressive tax, and displays the employee’s final net salary.

Case 5: Multi Plan Telecommunications Cost Optimization

Problem Statement

Evaluate mobile plans against usage, validity, and budget. Calculate excess usage charges and identify the most economical eligible plan.

Objective

Find the lowest cost plan that meets the required validity and budget.

Assumptions

- Each plan has a monthly fee, included minutes, included data, validity, and extra usage rates.

- Excess minutes and data are charged separately.

- A plan is eligible only if it meets the required validity and its calculated cost is within budget.

- Usage and costs cannot be negative.

Algorithm

1. Read user requirements: validity, expected call minutes, expected data, and budget.

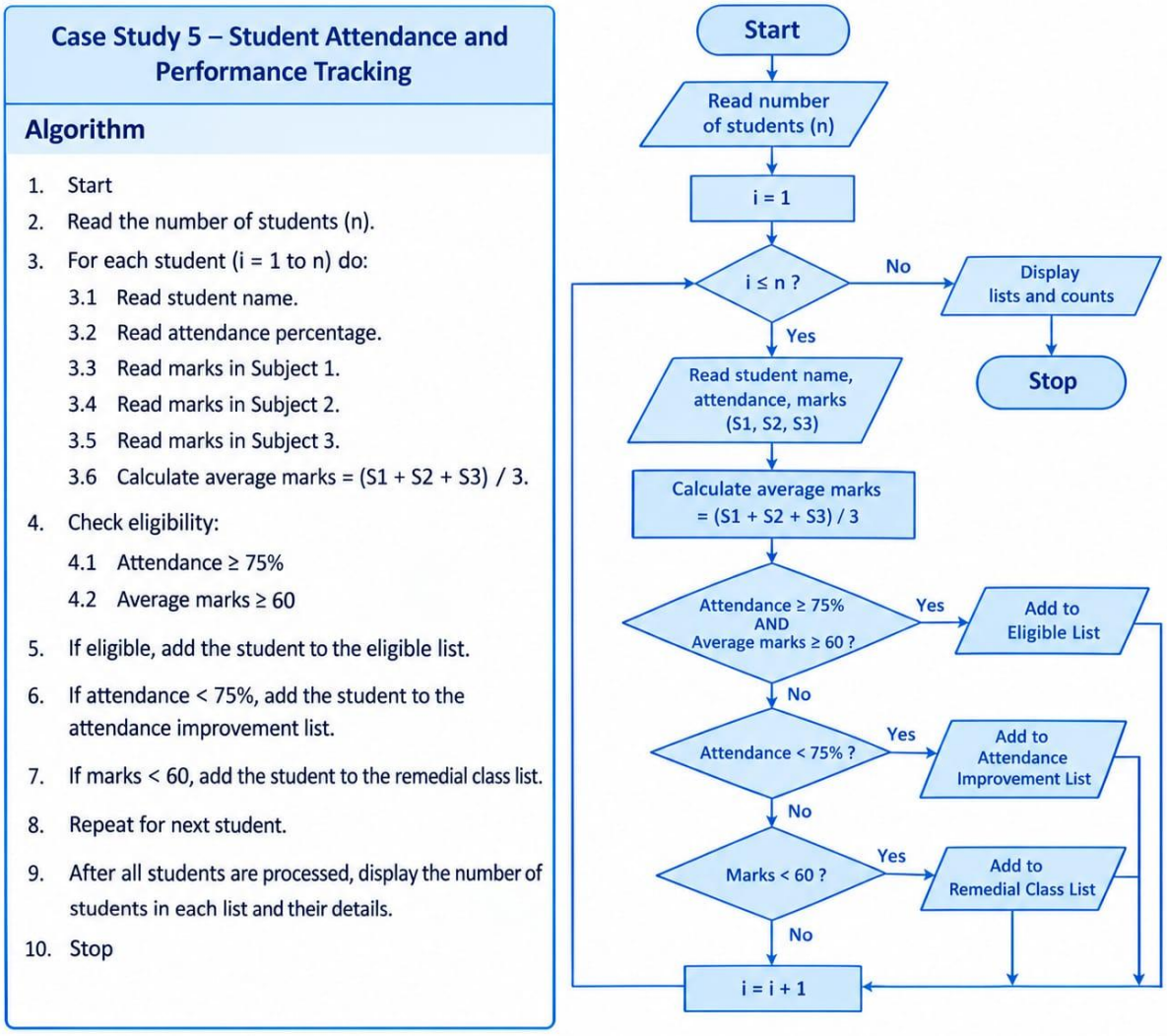
2. For each plan, read its fee, included usage, validity, and excess rates.

3. Ignore plans that do not meet the validity requirement.

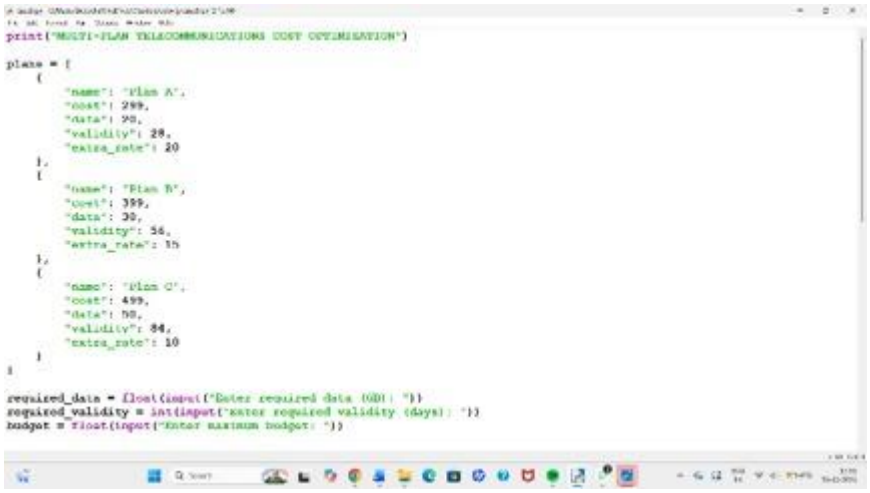
4. Calculate each remaining plan's total cost, including excess usage.

5. Keep plans within budget and display the least expensive one.

Flowchart



Pseudocode



```
print("MULTI-PLAN TELECOMMUNICATIONS COST OPTIMISATION")

plans = [
    {
        "name": "Plan A",
        "cost": 299,
        "data": 90,
        "validity": 28,
        "extra_rate": 20
    },
    {
        "name": "Plan B",
        "cost": 399,
        "data": 30,
        "validity": 56,
        "extra_rate": 15
    },
    {
        "name": "Plan C",
        "cost": 499,
        "data": 50,
        "validity": 84,
        "extra_rate": 10
    }
]

required_data = float(input("Enter required data (GB): "))
required_validity = int(input("enter required validity (days): "))
budget = float(input("Enter maximum budget: "))
```



```
        excess_data = 0
        extra_charge = 0

    total_cost = plan["cost"] + extra_charge

    if (plan["data"] >= required_data and
            plan["validity"] >= required_validity and
            total_cost <= budget):

        eligible_plans.append(
            (plan["name"], total_cost, plan["validity"])
        )

print("\nELIGIBLE PLANS")

if eligible_plans:
    for name, cost, validity in eligible_plans:
        print(f"{name} - Total Cost: ₹{cost:.2f}, "
              f"Validity: {validity} days")

    best_plan = min(eligible_plans, key=lambda x: x[1])

    print("\nMOST ECONOMICAL PLAN")
    print("Plan:", best_plan[0])
    print("Total Cost: ₹", round(best_plan[1], 2))
    print("Validity:", best_plan[2], "days")

else:
    print("No eligible plan found within the given budget.")
```



```
Python 3.13.14 (tags/v3.13.14:CD17597, Jun 10 2026, 13:63:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/case5.py ==========
MULTI-PLAN TELECOMMUNICATIONS COST OPTIMISATION
Enter required data (GB): 6
Enter required validity (days): 28
Enter maximum budget: 350

ELIGIBLE PLANS
Plan A - Total Cost: ₹299.00, Validity: 28 days

MOST ECONOMICAL PLAN
Plan: Plan A
Total Cost: ₹ 299
Validity: 28 days
>>>
```

INPUT requirements and number of plans

eligible_plans = empty list

```
FOR each plan
    INPUT plan fee, included usage, validity, excess rates
    IF plan validity >= required validity
        excess_cost = extra minutes cost + extra data cost
        total_cost = monthly fee + excess_cost
        IF total_cost <= budget
            ADD plan and total_cost to eligible_plans
        ENDIF
    ENDIF
ENDFOR
```

IF eligible_plans is empty

    DISPLAY no eligible plan

ELSE

    DISPLAY eligible plan with minimum total cost

ENDIF

Python Code

```
required_days = int(input("Required validity in days: "))
minutes_used = float(input("Expected call minutes: "))
data_used = float(input("Expected data usage in GB: "))
```

```
budget = float(input("Monthly budget: "))
count = int(input("Number of plans: "))

eligible = []

for i in range(count):
    print(f"\nPlan {i + 1}")
    name = input("Plan name: ")
    fee = float(input("Monthly fee: "))
    included_minutes = float(input("Included call minutes: "))
    included_data = float(input("Included data in GB: "))
```

```
    validity = int(input("Validity in days: "))
    extra_minute_rate = float(input("Cost per extra minute: "))
    extra_gb_rate = float(input("Cost per extra GB: "))

    values = [fee, included_minutes, included_data,
              extra_minute_rate, extra_gb_rate]

    if min(values) < 0 or validity < 0:
        print("Invalid plan values; plan ignored.")
```

```
        continue

    if validity >= required_days:
        extra_minutes = max(0,
minutes_used - included_minutes)
        extra_data = max(0, data_used -
included_data)
        total_cost = fee + extra_minutes *
extra_minute_rate
        total_cost += extra_data *
extra_gb_rate

        if total_cost <= budget:
            eligible.append([name,
total_cost])
```

```
if eligible:
    best = min(eligible, key=lambda plan: plan[1])
    print(f"\nMost economical eligible plan: {best[0]}")
    print(f"Estimated monthly cost: {best[1]:.2f}")
else:
    print("\nNo plan meets both validity and budget requirements.")
```

Test Case and Expected Output

Requirements: 30 days, 200 call minutes, 5 GB, budget 500. Plan Alpha costs 300, includes 250 minutes and 6 GB, and is valid for 30

days. Plan Beta costs 250, includes 100 minutes and 3 GB, charges 1 per extra minute and 50 per extra GB, and is valid for 30 days.

Most economical eligible plan: Alpha

Estimated monthly cost: 300.00

Conclusion

The program estimates the full cost of each qualifying plan, including excess usage, and selects the lowest cost plan within the budget.

Case 6: Railway Fare and Reservation Calculator

Problem Statement

Calculate fares for multiple passengers using distance slabs, travel class, age concessions, reservation fees, and cancellation penalties.

Objective

Calculate each passenger’s fare, total reservation cost, and refund after cancellation.

Assumptions

- Sleeper fare: 1 per kilometre; AC fare: 2 per kilometre.

- Children under 5 travel free but pay the reservation fee.

- Passengers aged 60 or over receive a 20% fare concession.

- Reservation fee: 30 per passenger.

- Cancellation penalty: 20% of the total fare, including reservation fees.

Algorithm

1. Read journey distance, travel class, and passenger count.

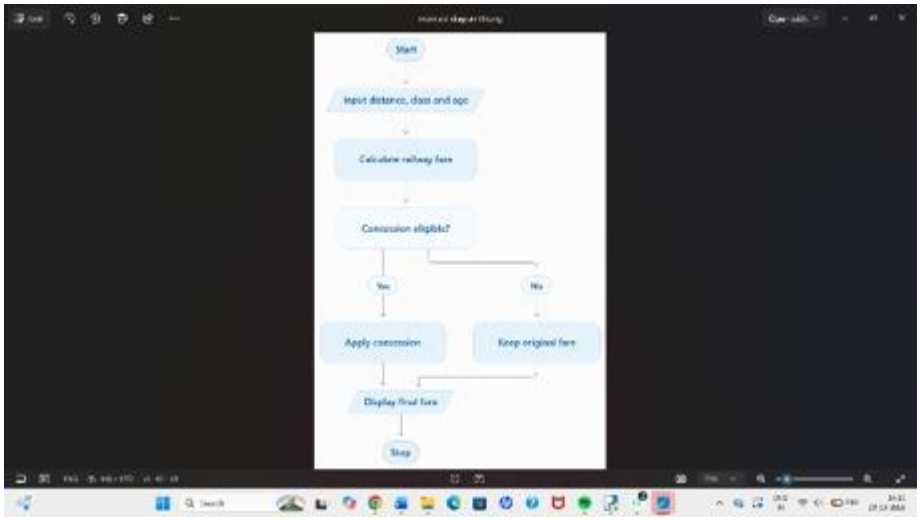
2. Calculate the fare rate for the selected class.

3. For each passenger, calculate fare after any age concession.

4. Add the reservation fee and sum the total.

5. If the user cancels, calculate the penalty and refund.

Flowchart



Pseudocode

case6.py - C:/Users/Deepthi/OneDrive/Desktop/activity/case6.py (3.13.14)

File Edit Format Run Options Window Help

```
print("RAILWAY FARE AND RESERVATION CALCULATOR")

n = int(input("Enter number of passengers: "))

total_revenue = 0

for i in range(n):
    print(f"\nPassenger {i + 1}")

    name = input("Enter passenger name: ")
    age = int(input("Enter age: "))
    distance = float(input("Enter travel distance (km): "))

    travel_class = input(
        "Enter class (Sleeper/AC3/AC2): "
    ).lower()

    cancelled = input(
        "Is the ticket cancelled? (yes/no): "
    ).lower()

    # Distance-based fare
    if distance <= 100:
        rate = 1.50
    elif distance <= 300:
        rate = 1.25
    else:
        rate = 1.00

    fare = distance * rate
```

Ln: 78 Col: 0

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

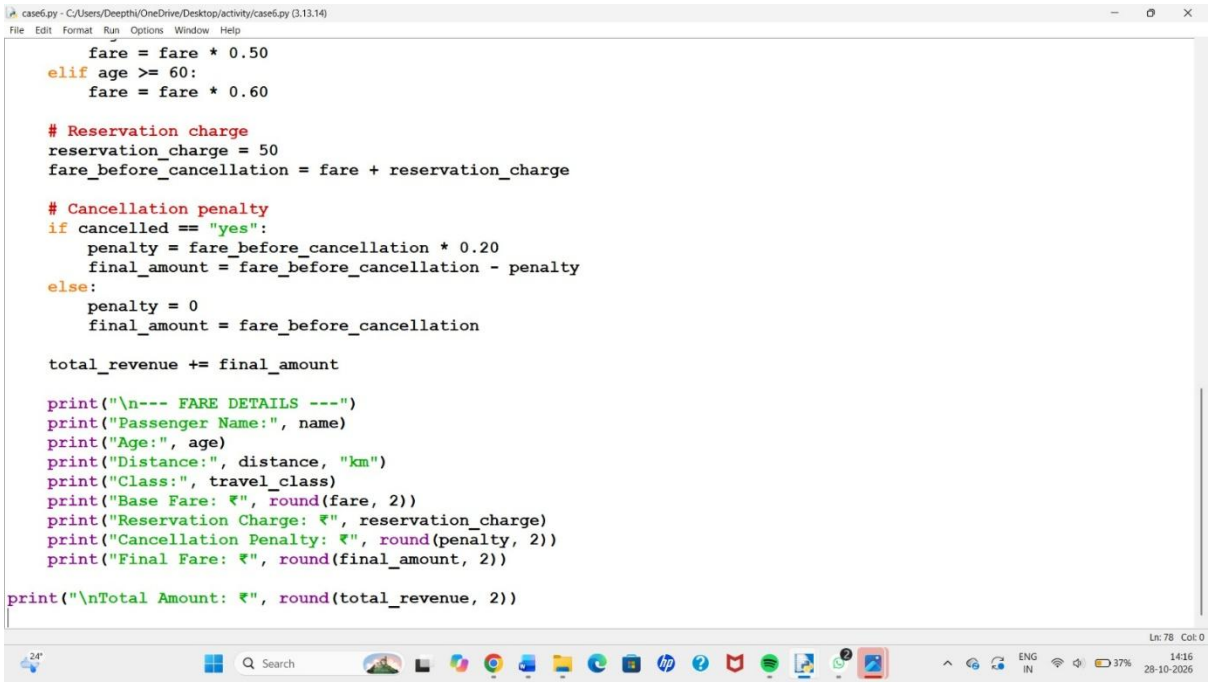
```
--- FARE DETAILS ---
Passenger Name: lavanya
Age: 45
Distance: 90.0 km
Class: sleeper
Base Fare: ₹ 135.0
Reservation Charge: ₹ 50
Cancellation Penalty: ₹ 37.0
Final Fare: ₹ 148.0

Passenger 2
Enter passenger name: kavya
Enter age: 67
Enter travel distance (km): 89
Enter class (Sleeper/AC3/AC2): ac3
Is the ticket cancelled? (yes/no): no

--- FARE DETAILS ---
Passenger Name: kavya
Age: 67
Distance: 89.0 km
Class: ac3
Base Fare: ₹ 120.15
Reservation Charge: ₹ 50
Cancellation Penalty: ₹ 0
Final Fare: ₹ 170.15

Total Amount: ₹ 318.15
>>>
```

Ln: 43 Col: 0



```
        fare = fare * 0.50
    elif age >= 60:
        fare = fare * 0.60

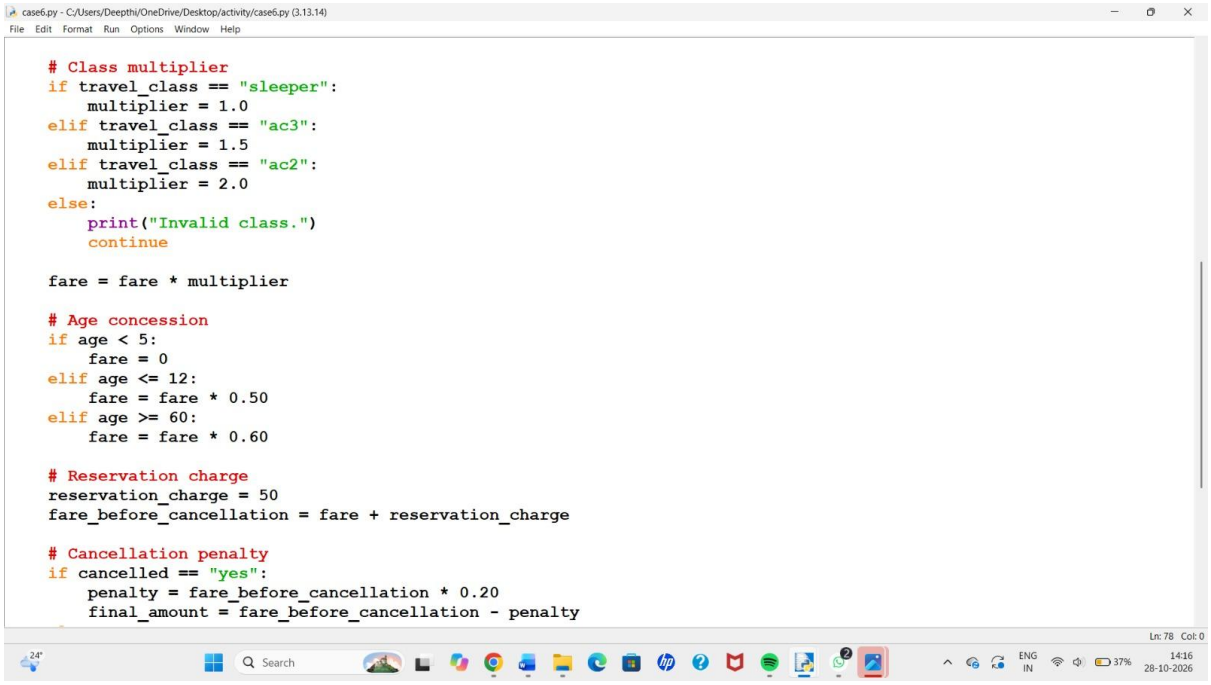
    # Reservation charge
    reservation_charge = 50
    fare_before_cancellation = fare + reservation_charge

    # Cancellation penalty
    if cancelled == "yes":
        penalty = fare_before_cancellation * 0.20
        final_amount = fare_before_cancellation - penalty
    else:
        penalty = 0
        final_amount = fare_before_cancellation

    total_revenue += final_amount

    print("\n--- FARE DETAILS ---")
    print("Passenger Name:", name)
    print("Age:", age)
    print("Distance:", distance, "km")
    print("Class:", travel_class)
    print("Base Fare: ₹", round(fare, 2))
    print("Reservation Charge: ₹", reservation_charge)
    print("Cancellation Penalty: ₹", round(penalty, 2))
    print("Final Fare: ₹", round(final_amount, 2))

print("\nTotal Amount: ₹", round(total_revenue, 2))
```



```
    # Class multiplier
    if travel_class == "sleeper":
        multiplier = 1.0
    elif travel_class == "ac3":
        multiplier = 1.5
    elif travel_class == "ac2":
        multiplier = 2.0
    else:
        print("Invalid class.")
        continue

    fare = fare * multiplier

    # Age concession
    if age < 5:
        fare = 0
    elif age <= 12:
        fare = fare * 0.50
    elif age >= 60:
        fare = fare * 0.60

    # Reservation charge
    reservation_charge = 50
    fare_before_cancellation = fare + reservation_charge

    # Cancellation penalty
    if cancelled == "yes":
        penalty = fare_before_cancellation * 0.20
        final_amount = fare_before_cancellation - penalty
```

INPUT distance, class, passenger count

SET rate based on class

total = 0

```
FOR each passenger
    INPUT age
    fare = distance * rate
    IF age < 5
        fare = 0
    ELSE IF age >= 60
        fare = fare * 0.80
    ENDIF
    total = total + fare + reservation fee
ENDFOR

DISPLAY total
IF cancelled
```

```
    penalty = total * cancellation rate
    DISPLAY refund = total - penalty
ENDIF
```

Python Code

```
distance = float(input("Journey distance in km: "))
travel_class = input("Class (sleeper/ac): ").lower()
passenger_count = int(input("Number of passengers: "))

if distance < 0 or passenger_count <= 0:
    print("Invalid distance or passenger count.")
```

```
else:
    if travel_class == "sleeper":
        rate = 1.0
    elif travel_class == "ac":
        rate = 2.0
    else:
        rate = None

    if rate is None:
        print("Invalid class.")
    else:
        reservation_fee = 30
        total = 0
```

```
        for i in range(passenger_count):
            age = int(input(f"Passenger {i + 1} age: "))
            fare = distance * rate

            if age < 5:
                fare = 0
            elif age >= 60:
                fare *= 0.80

            passenger_total = fare + reservation_fee
            total += passenger_total
            print(f"Passenger {i + 1} fare: {passenger_total:.2f}")
```

```
    print(f"Total reservation cost: {total:.2f}")


    cancel = input("Cancel reservation? (yes/no): ").lower()
    if cancel == "yes":
        penalty = total * 0.20
        print(f"Cancellation penalty: {penalty:.2f}")
        print(f"Refund: {total - penalty:.2f}")
```

Test Case and Expected Output

Distance: 100 km; class: sleeper; two passengers aged 30 and 65. First

passenger pays 100 + 30. Second pays 80 + 30.

Passenger 1 fare: 130.00

Passenger 2 fare: 110.00

Total reservation cost: 240.00

Cancel reservation? (yes/no): yes

Cancellation penalty: 48.00

Refund: 192.00

Conclusion

The program calculates passenger fares based on distance, travel class, age concessions, reservation charges, and cancellation rules.

Case 7: Retail Billing with Multiple Discount Rules

Problem Statement

Process multiple products and quantities, apply category-specific taxes and threshold discounts, and generate an itemized invoice with a maximum concession limit.

Objective

Calculate each product’s taxable amount and tax, apply a total discount subject to a cap, and print the final invoice.

Assumptions

- Each product has a user-entered price, quantity, and tax rate.

- A 5% discount applies to the subtotal when it reaches the entered threshold.

- The total discount cannot exceed the entered maximum.

- Discount is distributed proportionally across items before tax.

Algorithm

1. Read each product’s details and calculate line totals.

2. Add line totals to get the subtotal.

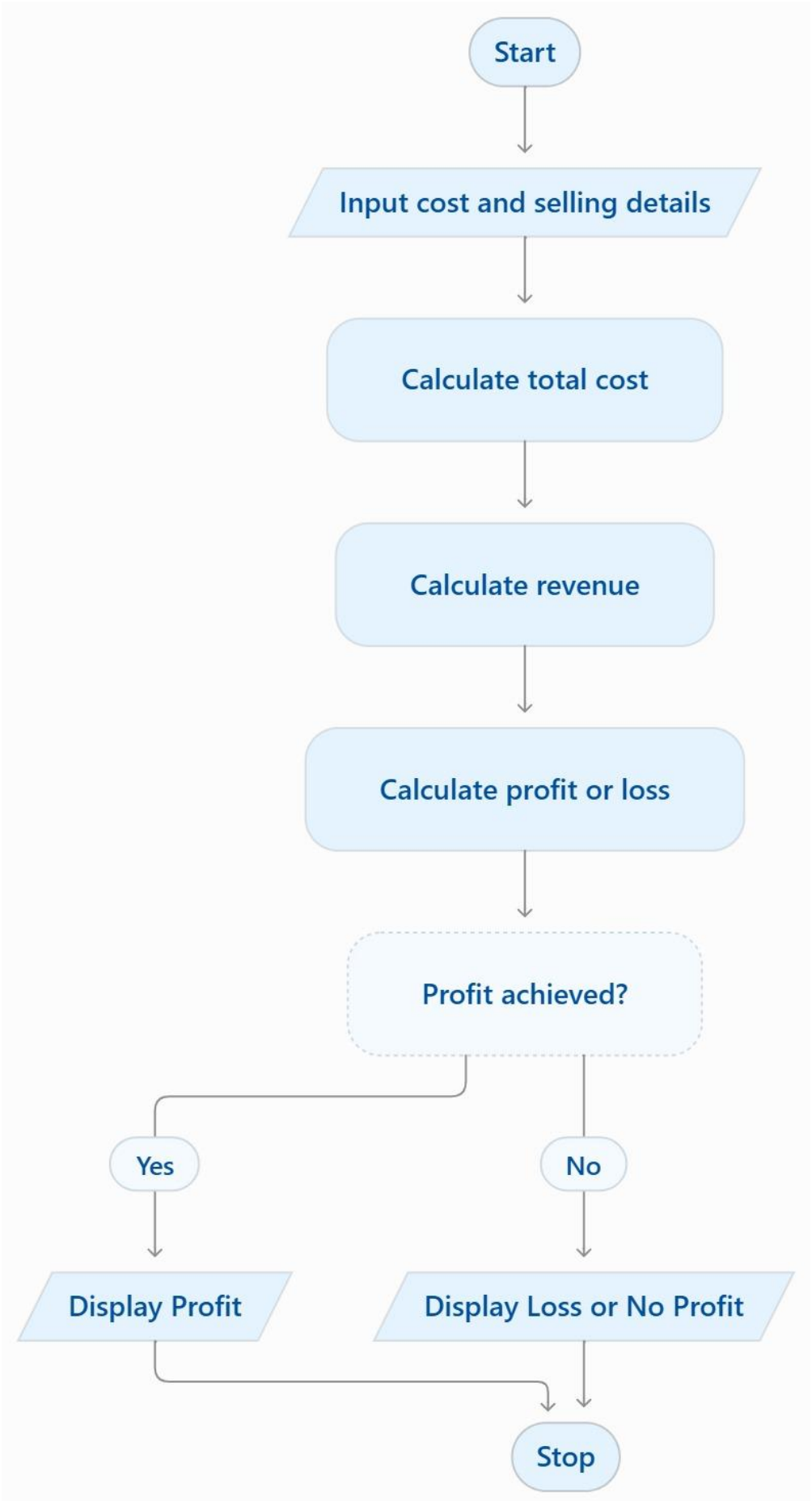
3. Check whether the subtotal reaches the discount threshold.

4. Calculate the discount and limit it to the maximum amount.

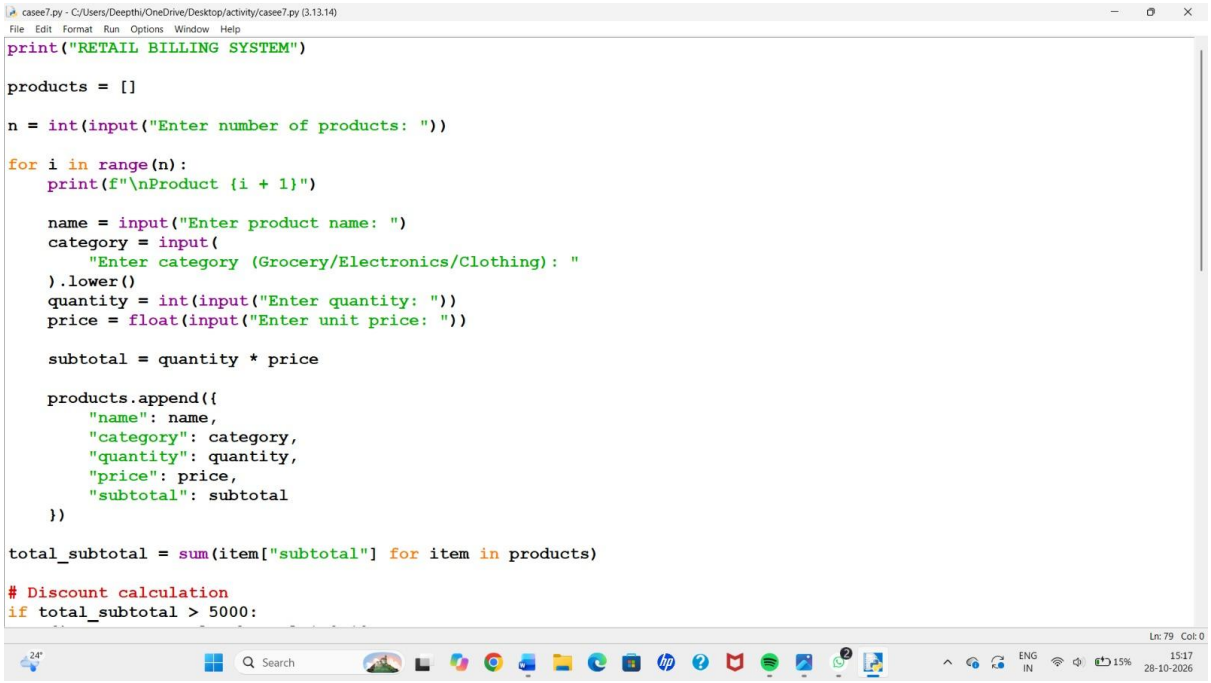
5. Allocate the discount proportionally to each product.

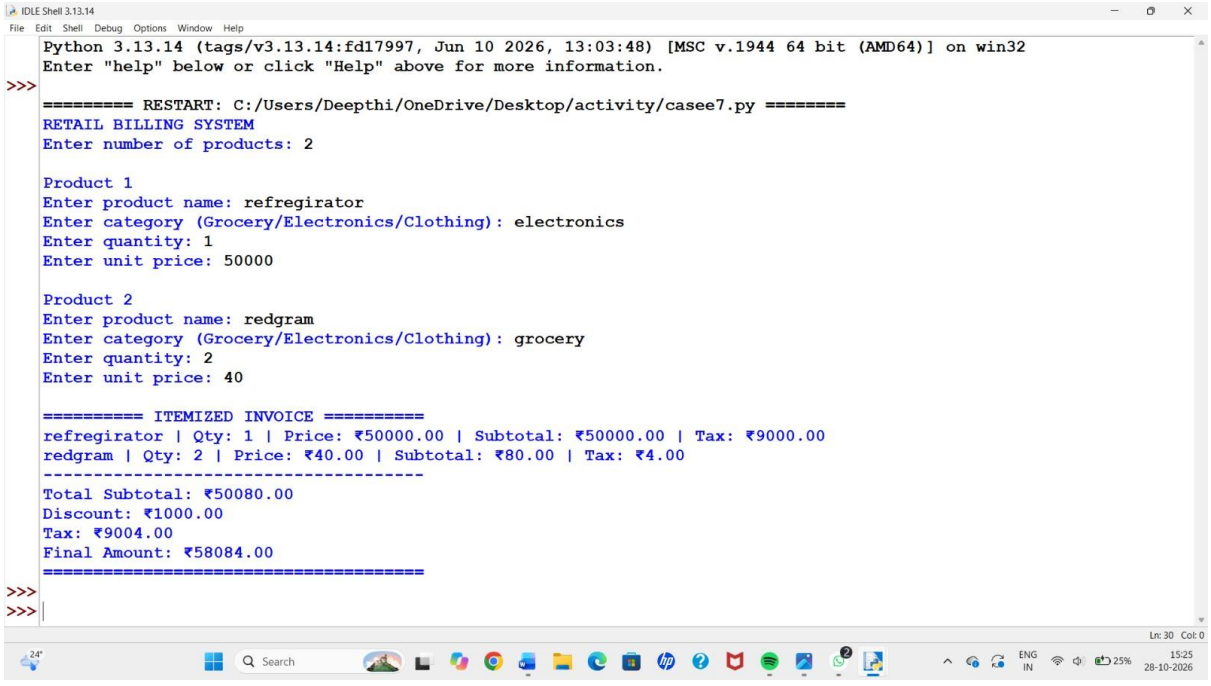
6. Calculate tax for each item and display the invoice totals.

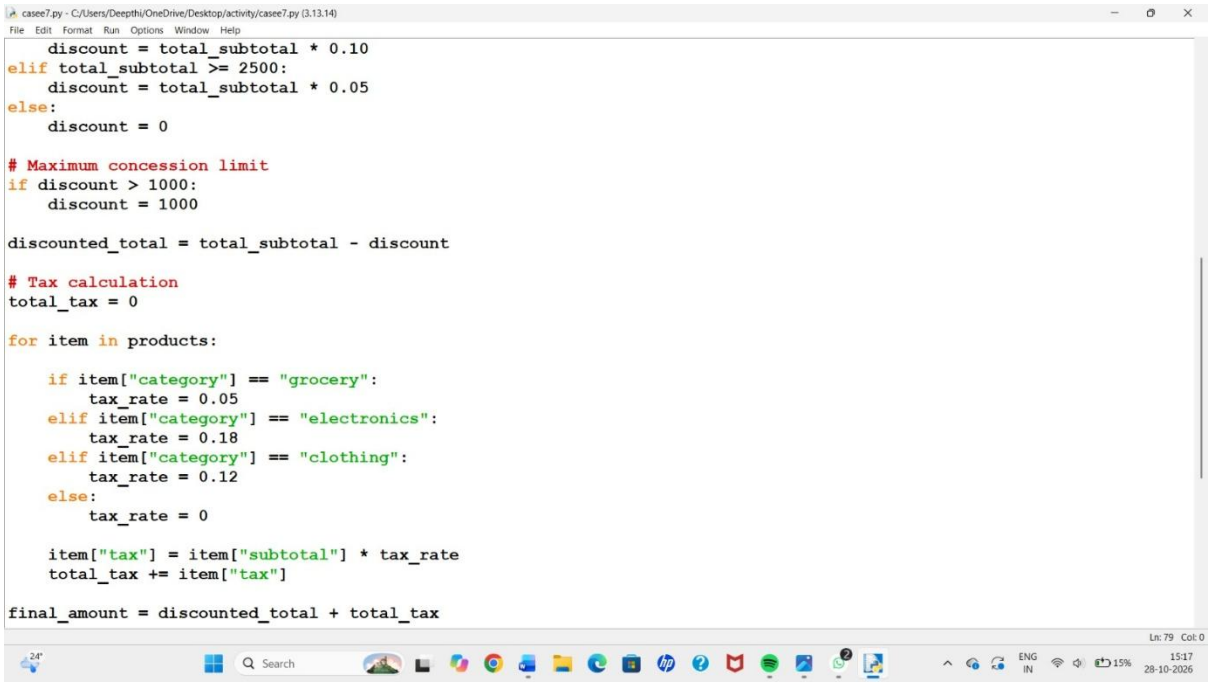
Flowchart


Start
Input cost and selling details
Calculate total cost
Calculate revenue
Calculate profit or loss
Profit achieved?
Yes
No
Display Profit
Display Loss or No Profit
Stop

Pseudocode


casee7.py - C:/Users/Deepthi/OneDrive/Desktop/activity/casee7.py (3.13.14)
File Edit Format Run Options Window Help
print("RETAIL BILLING SYSTEM")
products = []
n = int(input("Enter number of products: "))
for i in range(n):
print(f"\nProduct {i + 1}")
name = input("Enter product name: ")
category = input(
"Enter category (Grocery/Electronics/Clothing): "
).lower()
quantity = int(input("Enter quantity: "))
price = float(input("Enter unit price: "))
subtotal = quantity * price
products.append({
"name": name,
"category": category,
"quantity": quantity,
"price": price,
"subtotal": subtotal
})
total_subtotal = sum(item["subtotal"] for item in products)
Discount calculation
if total_subtotal > 5000:
Ln: 79 Col: 0


IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/casee7.py ========
RETAIL BILLING SYSTEM
Enter number of products: 2
Product 1
Enter product name: refregirator
Enter category (Grocery/Electronics/Clothing): electronics
Enter quantity: 1
Enter unit price: 50000
Product 2
Enter product name: redgram
Enter category (Grocery/Electronics/Clothing): grocery
Enter quantity: 2
Enter unit price: 40
========== ITEMIZED INVOICE ==========
refregirator | Qty: 1 | Price: ₹50000.00 | Subtotal: ₹50000.00 | Tax: ₹9000.00
redgram | Qty: 2 | Price: ₹40.00 | Subtotal: ₹80.00 | Tax: ₹4.00
Total Subtotal: ₹50080.00
Discount: ₹1000.00
Tax: ₹9004.00
Final Amount: ₹58084.00
>>>
>>>
Ln: 30 Col: 0



FOR each product

INPUT name, quantity, price, tax rate

line total = quantity * price

STORE product details

subtotal = subtotal + line total

ENDFOR

INPUT discount threshold and maximum discount

IF subtotal >= threshold

    discount = minimum(5% of subtotal, maximum discount)

ELSE

    discount = 0

ENDIF

FOR each product

    allocate discount proportionally

    calculate tax on discounted line total

    DISPLAY product, net amount, and tax

ENDFOR

DISPLAY subtotal, discount, tax total, and final total

Python Code

```
products = []
subtotal = 0

count = int(input("Number of products: "))

for i in range(count):
    print(f"\nProduct {i + 1}")
    name = input("Product name: ")
    quantity = int(input("Quantity: "))
```

```
    price = float(input("Unit price: "))
    tax_rate = float(input("Category tax rate (%): "))

    if quantity < 0 or price < 0 or tax_rate < 0:
        print("Invalid values; enter nonnegative quantity, price, and tax.")
        raise SystemExit

    line_total = quantity * price
    subtotal += line_total
    products.append([name, quantity, line_total, tax_rate])
```

```
threshold = float(input("\nDiscount threshold: "))
max_discount = float(input("Maximum discount: "))

if subtotal >= threshold:
    discount = min(subtotal * 0.05, max_discount)
else:
    discount = 0

tax_total = 0
print("\nItemized Invoice")
```

```
for name, quantity, line_total,
tax_rate in products:
    item_discount = discount *
line_total / subtotal if subtotal else 0
    taxable_amount = line_total -
item_discount
    tax = taxable_amount * tax_rate /
100
    tax_total += tax

    print(
        f"{name}: quantity={quantity}, "
        f"net={taxable_amount:.2f},
tax={tax:.2f}"
    )
```

final_total = subtotal - discount + tax_total

print(f"\nSubtotal: {subtotal:.2f}")

print(f"Discount: {discount:.2f}")

print(f"Tax total: {tax_total:.2f}")

print(f"Final total: {final_total:.2f}")

Test Case and Expected Output

Products: two notebooks at 100 each, taxed at 5%; one bag at 500, taxed at 10%. Discount threshold: 500; maximum discount: 100.

Itemized Invoice

Notebook: quantity=2, net=190.00, tax=9.50

Bag: quantity=1, net=475.00, tax=47.50

Subtotal: 700.00

Discount: 35.00

Tax total: 57.00

Final total: 722.00

Conclusion

The program calculates an itemized bill with proportional discounts, category-specific taxes, and a maximum discount limit.

Case 8: Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate material, labour, wastage, and machine costs; determine selling revenue and whether production meets a specified profit target.

Objective

Calculate total production cost, revenue, profit, and whether the profit target is met.

Assumptions

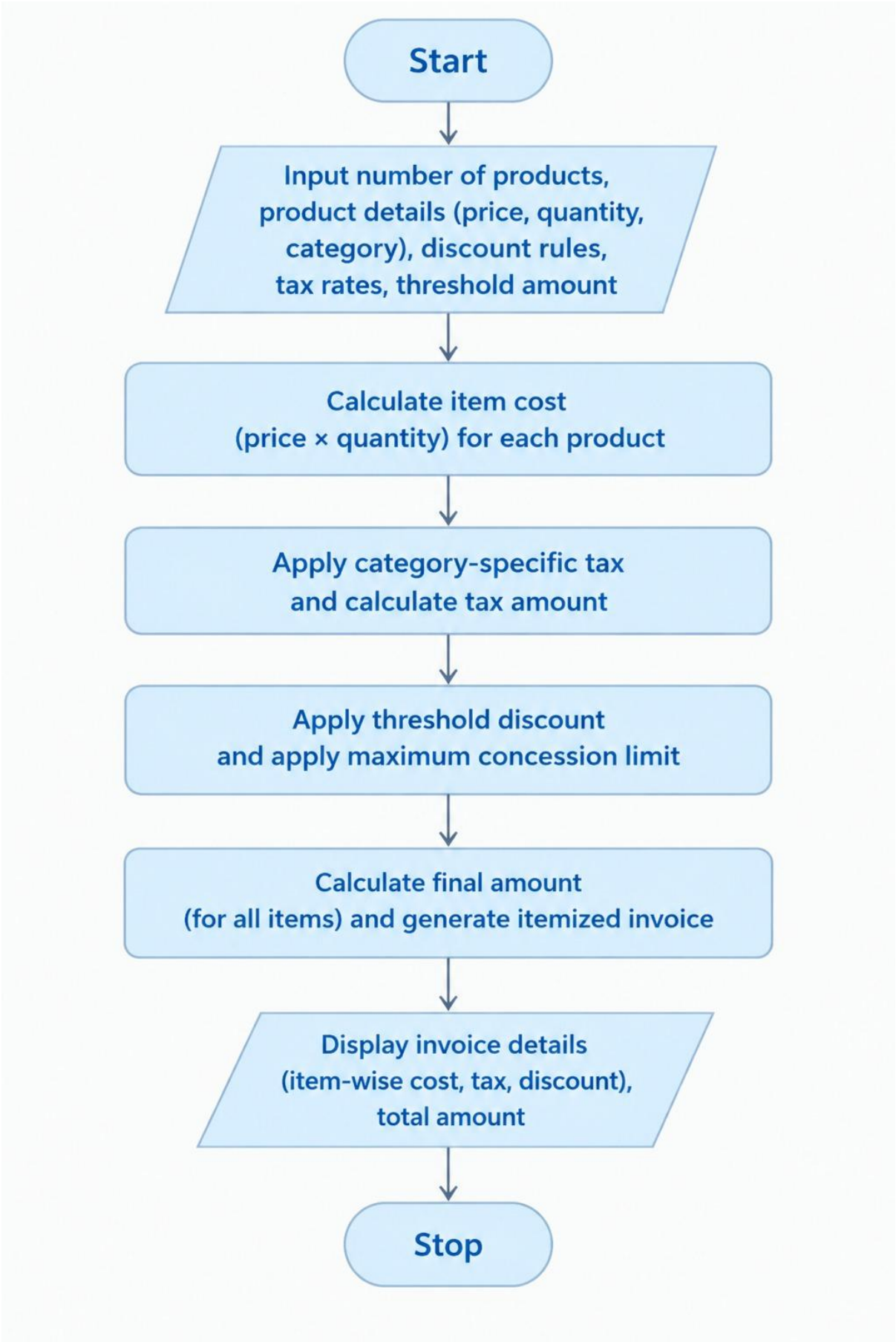
- Material cost is adjusted for the entered wastage percentage.
- Labour cost equals labour hours multiplied by the hourly rate.

- Machine cost equals machine hours multiplied by the machine rate.
- Revenue equals the number of units multiplied by the selling price per unit.

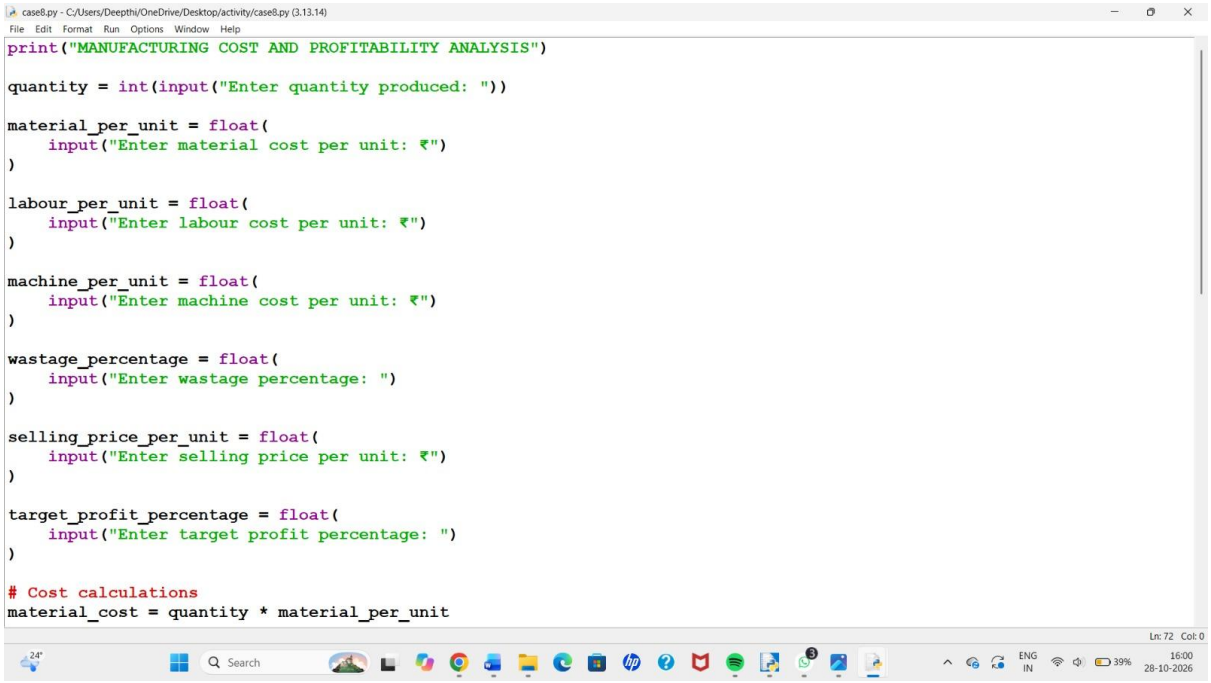
Algorithm

1. Read production quantity and cost details.
2. Add material wastage to the required material cost.
3. Calculate labour and machine costs.
4. Add all production costs.
5. Calculate revenue and profit.
6. Compare profit with the target and display the result.

Flowchart


Start
Input number of products, product details (price, quantity, category), discount rules, tax rates, threshold amount
Calculate item cost (price × quantity) for each product
Apply category-specific tax and calculate tax amount
Apply threshold discount and apply maximum concession limit
Calculate final amount (for all items) and generate itemized invoice
Display invoice details (item-wise cost, tax, discount), total amount
Stop

Pseudocode



```
case8.py - C:/Users/Deepthi/OneDrive/Desktop/activity/case8.py (3.13.14)
File  Edit  Format  Run  Options  Window  Help
print("MANUFACTURING COST AND PROFITABILITY ANALYSIS")

quantity = int(input("Enter quantity produced: "))

material_per_unit = float(
    input("Enter material cost per unit: ₹")
)

labour_per_unit = float(
    input("Enter labour cost per unit: ₹")
)

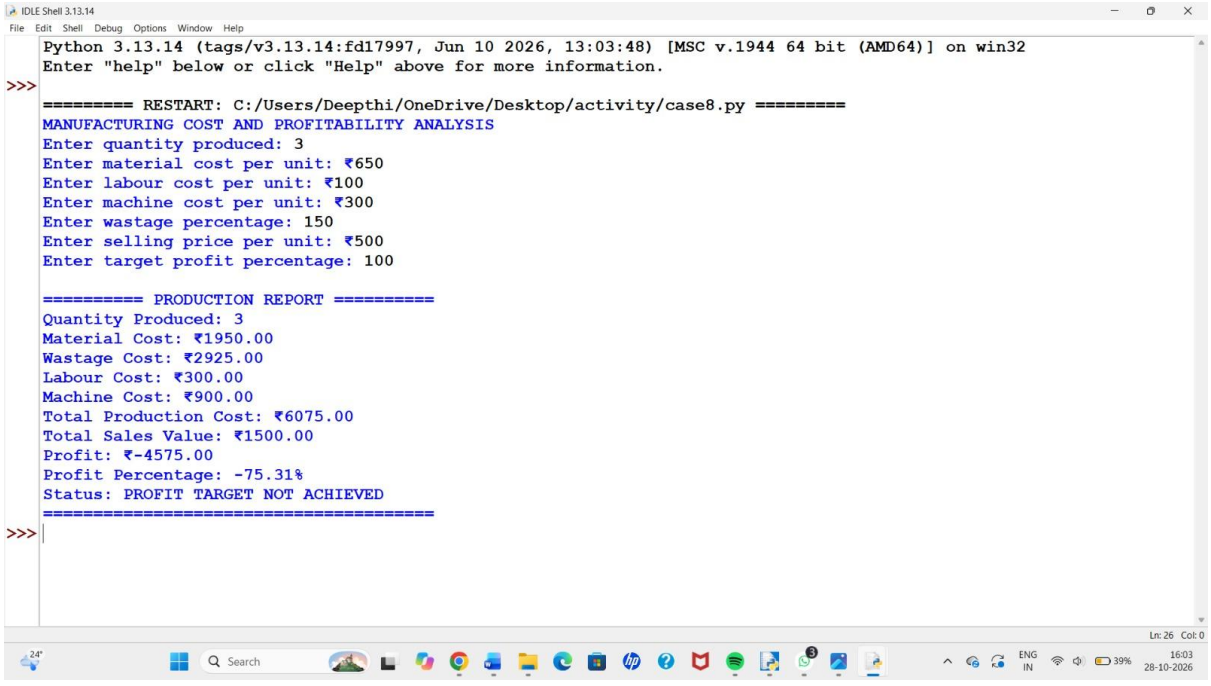
machine_per_unit = float(
    input("Enter machine cost per unit: ₹")
)

wastage_percentage = float(
    input("Enter wastage percentage: ")
)

selling_price_per_unit = float(
    input("Enter selling price per unit: ₹")
)

target_profit_percentage = float(
    input("Enter target profit percentage: ")
)

# Cost calculations
material_cost = quantity * material_per_unit
```



```
IDLE Shell 3.13.14
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/case8.py =========
MANUFACTURING COST AND PROFITABILITY ANALYSIS
Enter quantity produced: 3
Enter material cost per unit: ₹650
Enter labour cost per unit: ₹100
Enter machine cost per unit: ₹300
Enter wastage percentage: 150
Enter selling price per unit: ₹500
Enter target profit percentage: 100

========== PRODUCTION REPORT ==========
Quantity Produced: 3
Material Cost: ₹1950.00
Wastage Cost: ₹2925.00
Labour Cost: ₹300.00
Machine Cost: ₹900.00
Total Production Cost: ₹6075.00
Total Sales Value: ₹1500.00
Profit: ₹-4575.00
Profit Percentage: -75.31%
Status: PROFIT TARGET NOT ACHIEVED
========================================
>>>
```

INPUT units, material quantity and rate, wastage percentage

INPUT labour hours and rate, machine hours and rate

INPUT selling price per unit and profit target

material cost = quantity * rate * (1 + wastage / 100)

labour cost = labour hours * labour rate

machine cost = machine hours * machine rate

total cost = material cost + labour cost + machine cost

revenue = units * selling price

profit = revenue - total cost

IF profit >= target

    DISPLAY target achieved

ELSE

    DISPLAY target not achieved

ENDIF

Python Code

```
units = int(input("Number of units to produce: "))
material_quantity = float(input("Material quantity required: "))
material_rate = float(input("Material cost per unit: "))
```

```
wastage_percent =
float(input("Wastage percentage: "))
labour_hours = float(input("Labour
hours: "))
labour_rate = float(input("Labour
cost per hour: "))
machine_hours =
float(input("Machine hours: "))
machine_rate = float(input("Machine
cost per hour: "))
selling_price = float(input("Selling
price per finished unit: "))
profit_target = float(input("Profit
target: "))


values = [
```

```
    units, material_quantity,
material_rate, wastage_percent,
    labour_hours, labour_rate,
machine_hours, machine_rate,
    selling_price, profit_target
]

if min(values) < 0:
    print("Invalid: enter nonnegative
values.")
else:
    material_cost = (
        material_quantity * material_rate
* (1 + wastage_percent / 100)
    )
```

```
    labour_cost = labour_hours *
labour_rate
    machine_cost = machine_hours *
machine_rate
    total_cost = material_cost +
labour_cost + machine_cost
    revenue = units * selling_price
    profit = revenue - total_cost


    print(f"\nMaterial cost:
{material_cost:.2f}")
    print(f"Labour cost:
{labour_cost:.2f}")
    print(f"Machine cost:
{machine_cost:.2f}")
```

```
    print(f"Total production cost: {total_cost:.2f}")
    print(f"Revenue: {revenue:.2f}")
    print(f"Profit: {profit:.2f}")

    if profit >= profit_target:
        print("Production meets the profit target.")
    else:
        print("Production does not meet the profit target.")
```

Test Case and Expected Output

Produce 100 units. Material: 100 units at 10 each, with 5% wastage. Labour: 20 hours at 100/hour.

Machine: 10 hours at 50/hour. Selling price: 20/unit. Target profit: 500.

Material cost: 1050.00

Labour cost: 2000.00

Machine cost: 500.00

Total production cost: 3550.00

Revenue: 2000.00

Profit: -1550.00

Production does not meet the profit target.

Conclusion

The program totals the stated production costs, compares them with expected sales revenue, and

checks the result against the profit target.

Case 9: Multi Currency Travel Budget Analysis

Problem Statement

Convert travel expenses using supplied exchange rates, include conversion fees, compare planned spending with a budget, and identify the largest expense category.

Objective

Calculate total travel spending in the home currency, determine whether spending is within budget, and find the largest expense category.

Assumptions

- The exchange rate is entered as home currency per one unit of foreign currency.

- Conversion fee is a percentage of the converted amount.

- Expense amounts and fees cannot be negative.

- The largest category is based on the final converted amount including the fee.

Algorithm

1. Read each expense category, foreign amount, exchange rate, and conversion fee.

2. Convert the expense to the home currency.

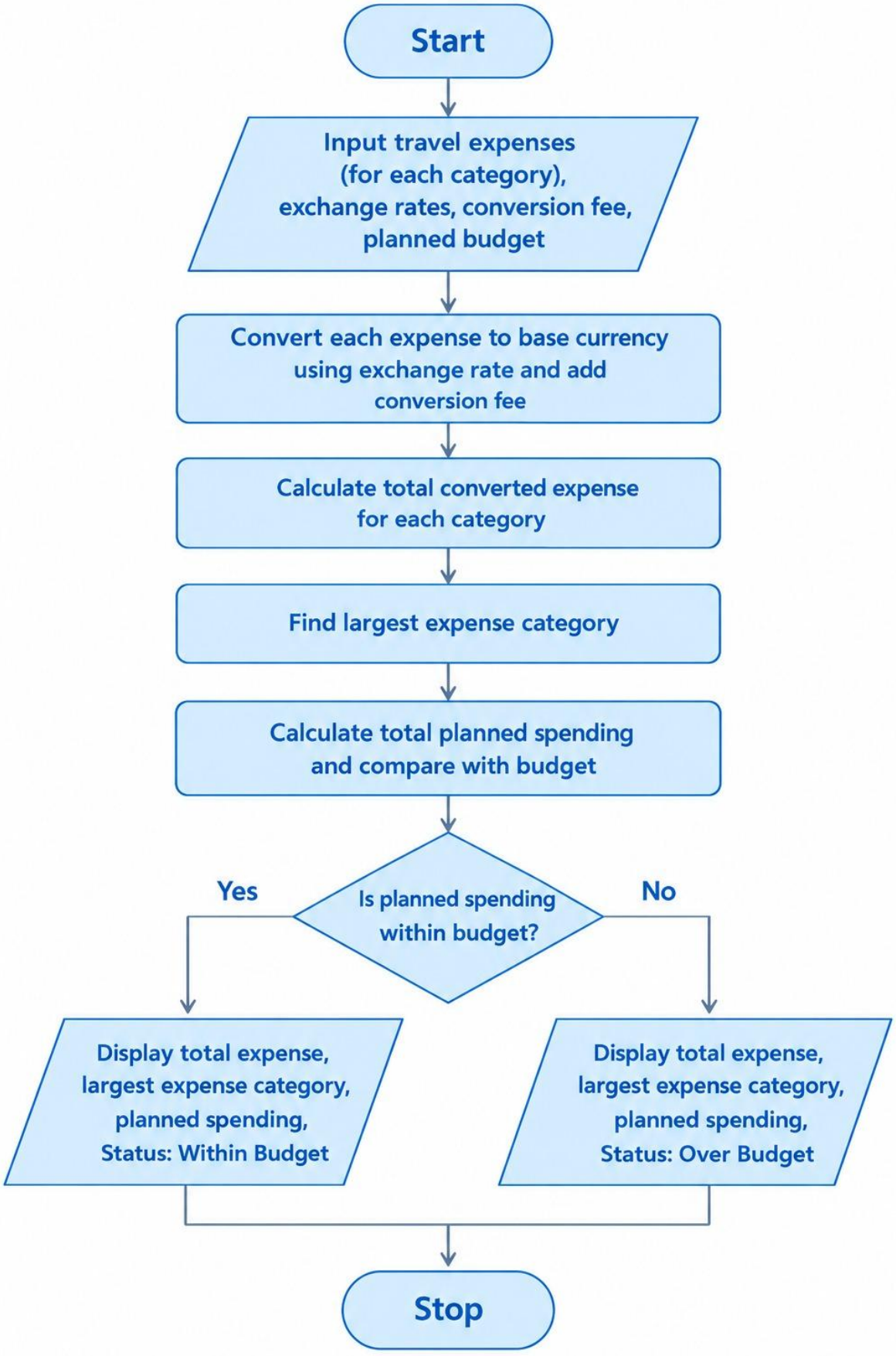
3. Add the conversion fee.

4. Sum all converted expenses.

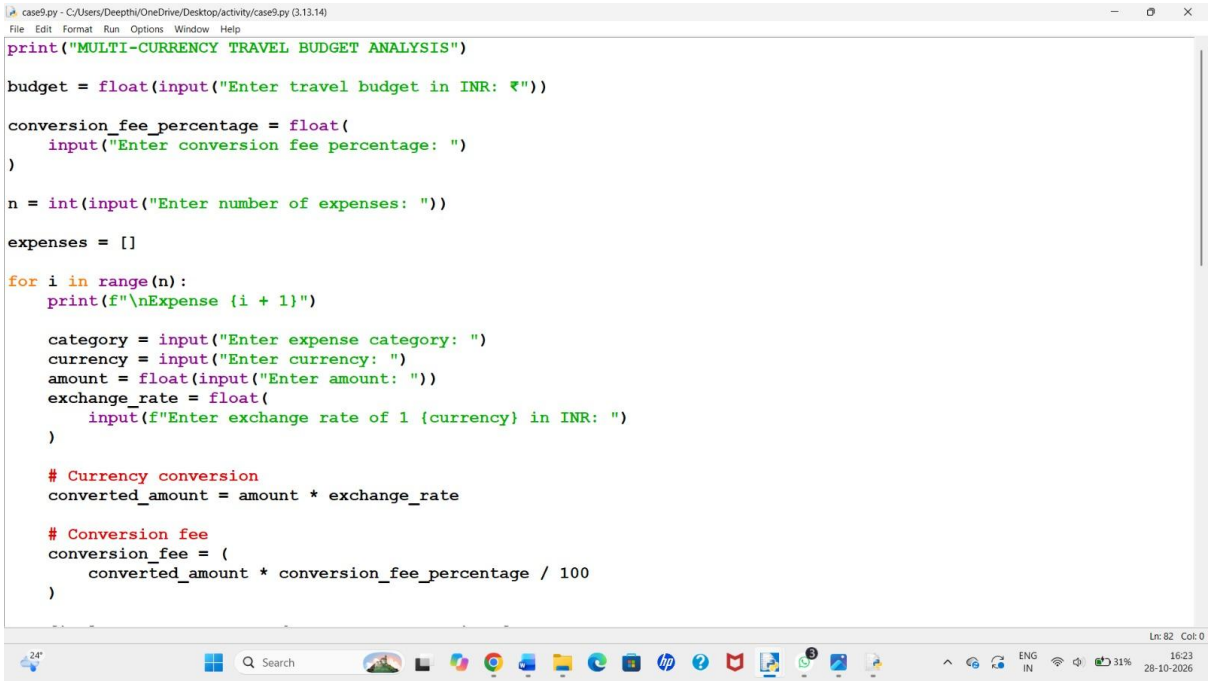
5. Compare the total with the budget.

6. Display the largest expense category.

Flowchart


Start
Input travel expenses (for each category), exchange rates, conversion fee, planned budget
Convert each expense to base currency using exchange rate and add conversion fee
Calculate total converted expense for each category
Find largest expense category
Calculate total planned spending and compare with budget
Is planned spending within budget?
Yes
No
Display total expense, largest expense category, planned spending, Status: Within Budget
Display total expense, largest expense category, planned spending, Status: Over Budget
Stop

Pseudocode



```
print("MULTI-CURRENCY TRAVEL BUDGET ANALYSIS")

budget = float(input("Enter travel budget in INR: ₹"))

conversion_fee_percentage = float(
    input("Enter conversion fee percentage: ")
)

n = int(input("Enter number of expenses: "))

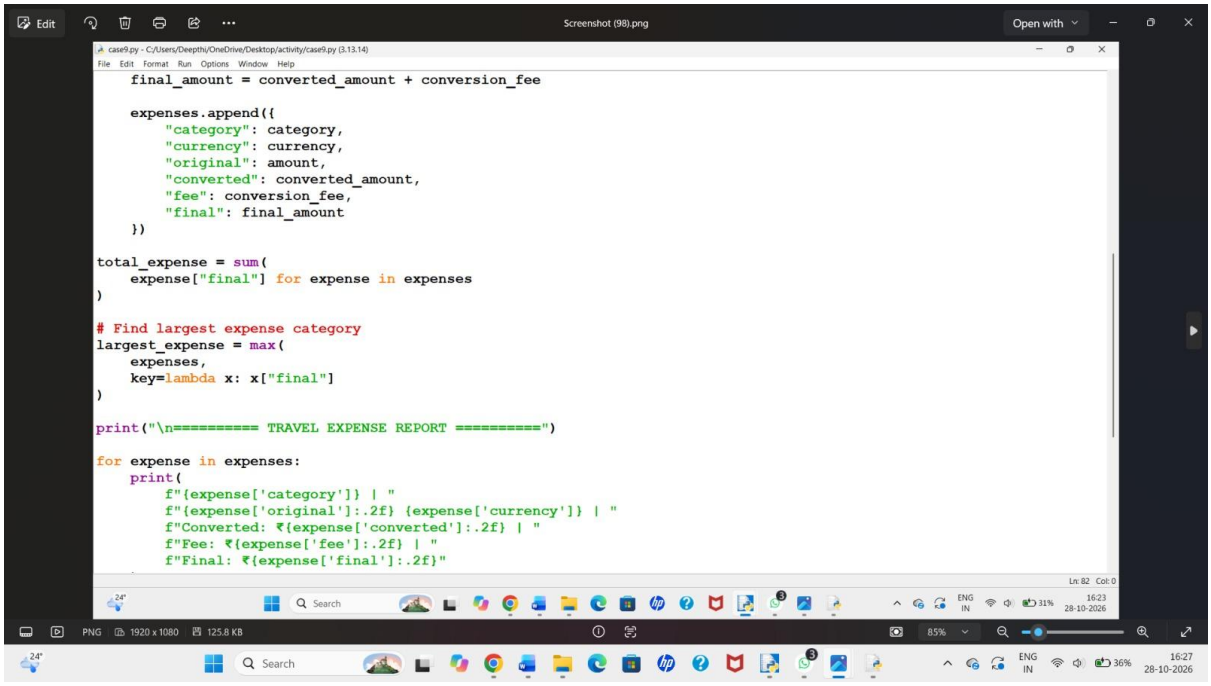
expenses = []

for i in range(n):
    print(f"\nExpense {i + 1}")

    category = input("Enter expense category: ")
    currency = input("Enter currency: ")
    amount = float(input("Enter amount: "))
    exchange_rate = float(
        input(f"Enter exchange rate of 1 {currency} in INR: ")
    )

    # Currency conversion
    converted_amount = amount * exchange_rate

    # Conversion fee
    conversion_fee = (
        converted_amount * conversion_fee_percentage / 100
    )
```



```
    final_amount = converted_amount + conversion_fee

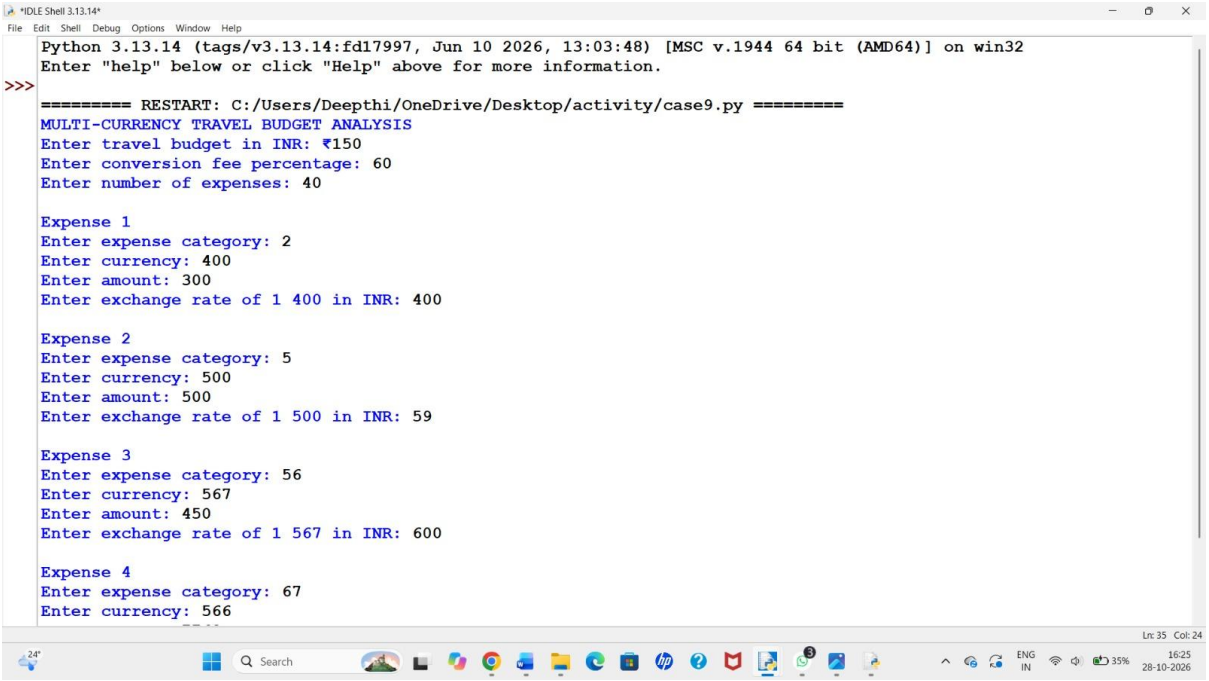
    expenses.append({
        "category": category,
        "currency": currency,
        "original": amount,
        "converted": converted_amount,
        "fee": conversion_fee,
        "final": final_amount
    })

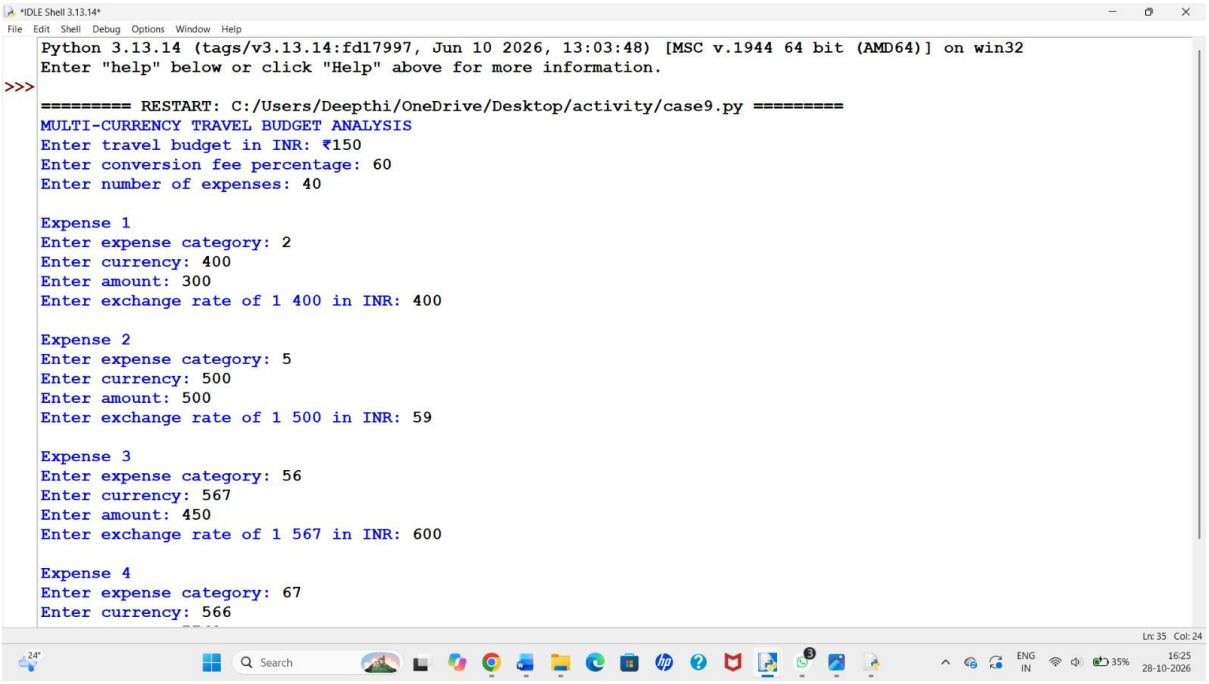
total_expense = sum(
    expense["final"] for expense in expenses
)

# Find largest expense category
largest_expense = max(
    expenses,
    key=lambda x: x["final"]
)

print("\n========== TRAVEL EXPENSE REPORT ==========")

for expense in expenses:
    print(
        f"{expense['category']} | "
        f"{expense['original']:.2f} {expense['currency']} | "
        f"Converted: ₹{expense['converted']:.2f} | "
        f"Fee: ₹{expense['fee']:.2f} | "
        f"Final: ₹{expense['final']:.2f}"
```





INPUT number of expense categories

expenses = empty list

FOR each category

    INPUT category, foreign amount, exchange rate, fee percentage

    converted = foreign amount * exchange rate

    total cost = converted * (1 + fee percentage / 100)

    STORE category and total cost

ENDFOR

INPUT budget

total spending = sum of all costs

largest expense = category with highest cost

DISPLAY costs, total, remaining budget, and largest category

Python Code

```
expenses = []
count = int(input("Number of expense categories: "))

for i in range(count):
    print(f"\nExpense {i + 1}")
    category = input("Category: ")
    foreign_amount = float(input("Amount in foreign currency: "))
    exchange_rate = float(input("Home currency per foreign unit: "))
```

```
    fee_percent =
float(input("Conversion fee (%): "))

    if min(foreign_amount,
exchange_rate, fee_percent) < 0:
        print("Invalid: amounts, rates,
and fees cannot be negative.")
        raise SystemExit

    converted_amount =
foreign_amount * exchange_rate
    total_cost = converted_amount * (1
+ fee_percent / 100)
    expenses.append([category,
total_cost])
```

```
budget = float(input("\nTravel budget in home currency: "))
if budget < 0:
    print("Budget cannot be negative.")
    raise SystemExit

total_spending = sum(item[1] for item in expenses)

for category, cost in expenses:
    print(f"{category}: {cost:.2f}")

print(f"\nTotal spending: {total_spending:.2f}")
```

```
print(f"Budget remaining: {budget - total_spending:.2f}")
print("Within budget." if total_spending <= budget else "Over budget.")

if expenses:
    largest = max(expenses, key=lambda item: item[1])
    print(f"Largest expense: {largest[0]} ({largest[1]:.2f})")
else:
    print("No expense categories were entered.")
```

Test Case and Expected Output

Hotel: 100 foreign units at 80 home currency per unit, with 2% fee. Food: 50 foreign units at the same rate, with 1% fee. Budget: 13,000.

Hotel: 8160.00

Food: 4040.00

Total spending: 12200.00

Budget remaining: 800.00

Within budget.

Largest expense: Hotel (8160.00)

Conclusion

The program converts each expense, includes its fee, compares total

spending with the budget, and reports the largest cost category.

Case 10: Campus Event Financial Planning

Problem Statement

Calculate event costs for venue, equipment, catering, transport, and certificates. Determine participant contributions, sponsorship shortfall, and the minimum registrations needed to break even.

Objective

Calculate total event costs and funding, identify any shortfall, and estimate the break even registration count.

Assumptions

- Venue, equipment, and transport are fixed costs.

- Catering and certificate costs are per participant.

- Each participant pays the entered registration fee.

- Sponsorship is a fixed contribution.

- Break even registrations are calculated from total fixed costs after sponsorship and the contribution remaining after per participant costs.

Algorithm

1. Read fixed and per participant costs.

2. Read participant count, registration fee, and sponsorship.

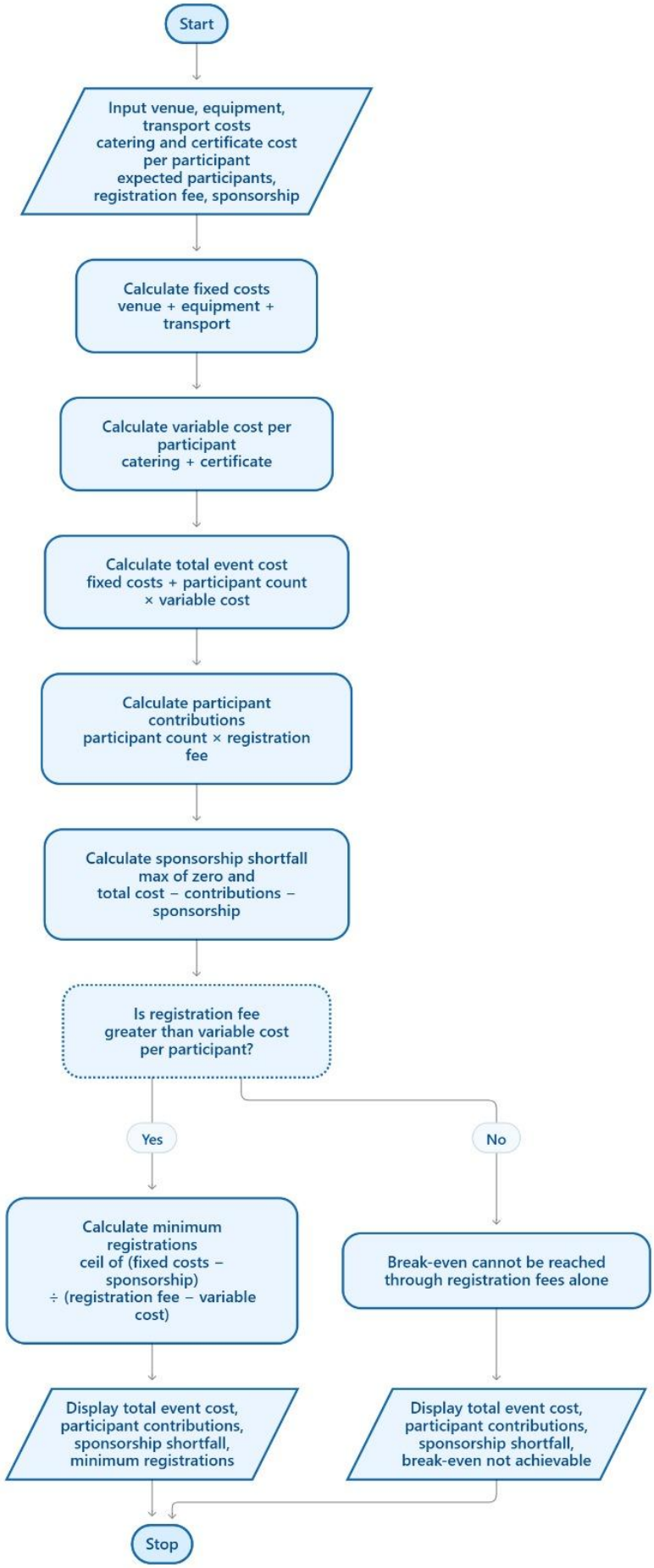
3. Calculate the event’s total cost and participant contributions.

4. Calculate total funding and any shortfall.

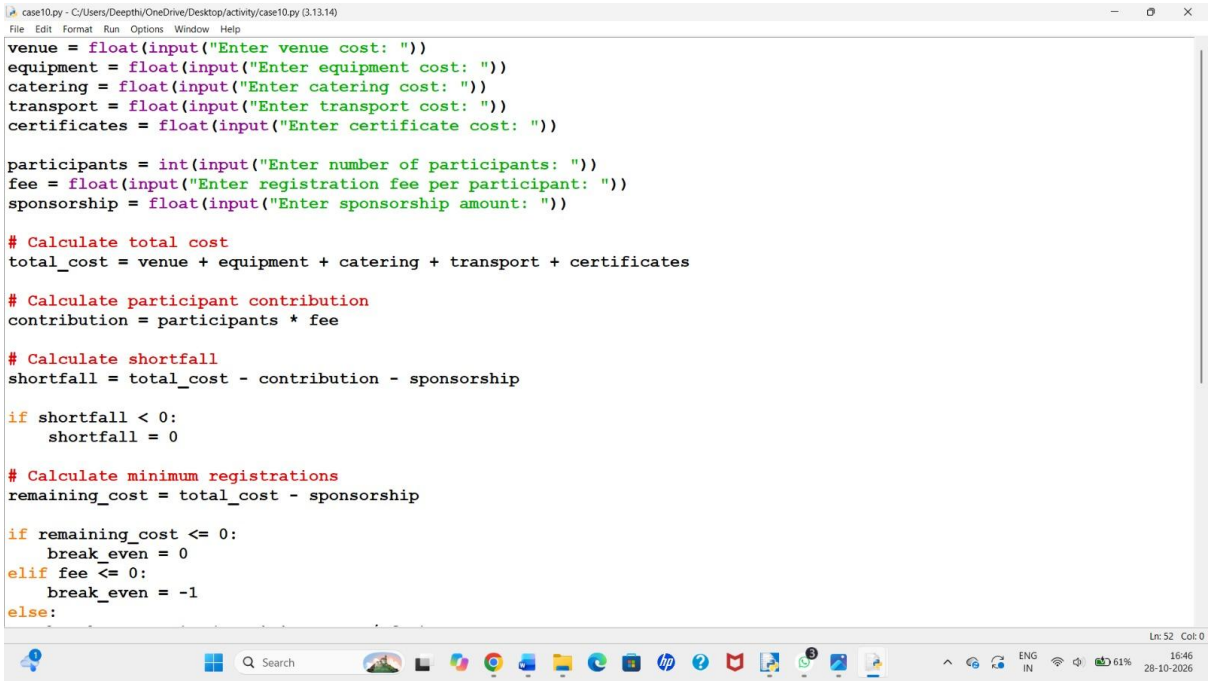
5. Calculate the minimum registrations needed to cover costs.

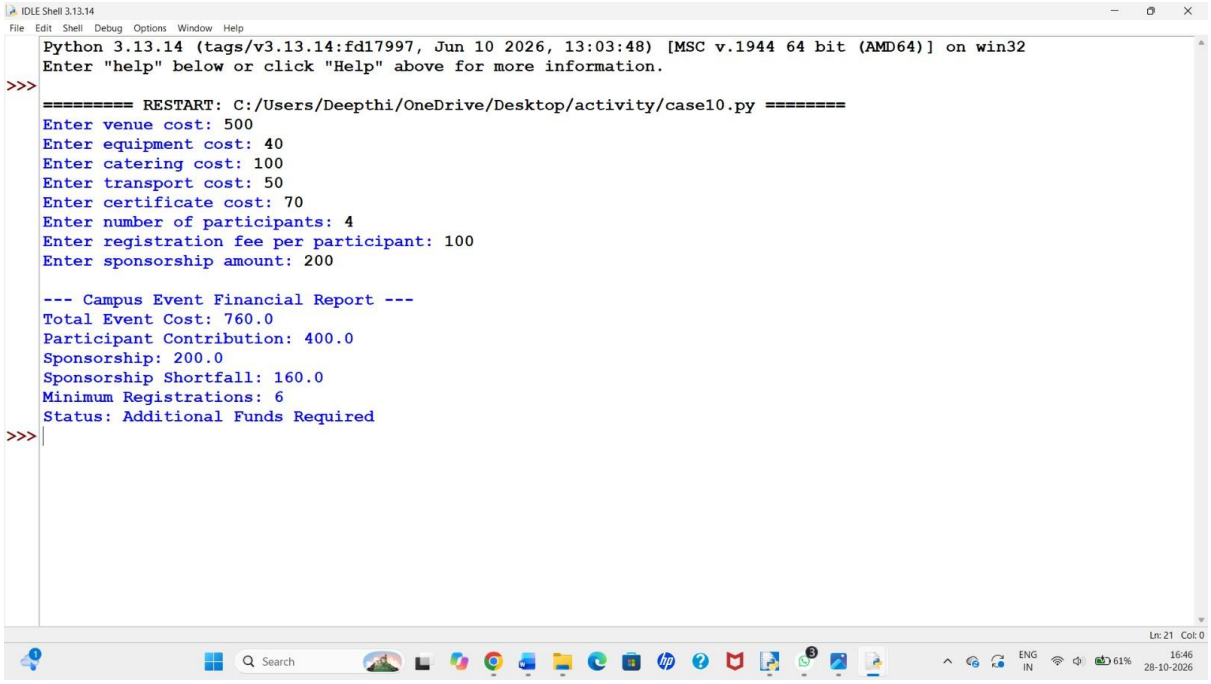
6. Display the results.

Flowchart


Start
Input venue, equipment, transport costs catering and certificate cost per participant expected participants, registration fee, sponsorship
Calculate fixed costs venue + equipment + transport
Calculate variable cost per participant catering + certificate
Calculate total event cost fixed costs + participant count × variable cost
Calculate participant contributions participant count × registration fee
Calculate sponsorship shortfall max of zero and total cost − contributions − sponsorship
Is registration fee greater than variable cost per participant?
Yes
No
Calculate minimum registrations ceil of (fixed costs − sponsorship) ÷ (registration fee − variable cost)
Break-even cannot be reached through registration fees alone
Display total event cost, participant contributions, sponsorship shortfall, minimum registrations
Display total event cost, participant contributions, sponsorship shortfall, break-even not achievable
Stop

Pseudocode





INPUT venue, equipment, transport

INPUT catering and certificate costs per participant

INPUT expected participants, registration fee, sponsorship

fixed cost = venue + equipment + transport

variable cost per participant = catering + certificate

total cost = fixed cost + participants * variable cost per participant

contributions = participants * registration fee

total funding = contributions + sponsorship

shortfall = maximum(0, total cost - total funding)

IF registration fee <= variable cost per participant

    break even registrations = not achievable from registration alone

ELSE

    break even registrations =

        ceiling((fixed cost - sponsorship) /

            (registration fee - variable cost per participant))

ENDIF

DISPLAY costs, funding, shortfall, and break even count

Python Code

```
import math


venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
transport = float(input("Transport cost: "))
catering_per_person = float(input("Catering cost per participant: "))
certificate_per_person = float(input("Certificate cost per participant: "))
```

```
participants = int(input("Expected
number of participants: "))
registration_fee =
float(input("Registration fee per
participant: "))
sponsorship =
float(input("Sponsorship received: "))


values = [
    venue, equipment, transport,
catering_per_person,
    certificate_per_person,
participants, registration_fee,
sponsorship
]
```

```
if min(values) < 0:
    print("Invalid: costs, participant count, and funding cannot be negative.")
else:
    fixed_cost = venue + equipment + transport
    variable_cost = catering_per_person + certificate_per_person
    total_cost = fixed_cost + participants * variable_cost
    participant_contributions = participants * registration_fee
```

```
    total_funding =
participant_contributions +
sponsorship
    shortfall = max(0, total_cost -
total_funding)


amount_needed_from_registrations =
fixed_cost - sponsorship
    contribution_per_registration =
registration_fee - variable_cost


    if
amount_needed_from_registrations
<= 0:
        break_even = 0
```

```
    elif contribution_per_registration
<= 0:
        break_even = None
    else:
        break_even = math.ceil(

amount_needed_from_registrations /
contribution_per_registration
        )

    print(f"\nFixed costs:
{fixed_cost:.2f}")
    print(f"Total event cost:
{total_cost:.2f}")
```

```
    print(f"Participant contributions: {participant_contributions:.2f}")
    print(f"Sponsorship: {sponsorship:.2f}")
    print(f"Overall funding shortfall: {shortfall:.2f}")

    if break_even is None:
        print("Break even is not achievable from registration fees alone.")
    else:
        print(f"Minimum registrations to break even: {break_even}")
```

Test Case and Expected Output

Fixed costs: venue 5,000; equipment 1,000; transport 1,000. Catering and certificate costs are 100 per participant. Registration fee: 200. Sponsorship: 1,000. Expected participants: 50.

Fixed costs: 7000.00

Total event cost: 12000.00

Participant contributions: 10000.00

Sponsorship: 1000.00

Overall funding shortfall: 1000.00

Minimum registrations to break even: 60

Conclusion

The program calculates fixed and variable event costs, totals the

available funding, finds any shortfall, and estimates the registrations needed to break even.